

# Predicció de victòries a League of Legends

Autor: Benaia J. Heinze. NIU: 1634990

Data: 06/12/2025

## 1. Introducció

*League of Legends* (LoL) és un videojoc del gènere Multiplayer Online Battle Arena (MOBA) desenvolupat i publicat per Riot Games. És el videojoc més popular d'aquest gènere. El joc consisteix a jugar partides autocontingudes, és a dir, jugar una partida aïllades que no afecten directament les pròximes partides, entre dos equips diferents (Blau i Vermell) que han de destruir la base (Nexe) de l'equip contrari. Aquesta és l'única manera de guanyar la partida, assumint que cap equip es rendeixi, ja que és impossible quedar empatats.

Ara, es tracta d'un joc d'estratègia i, per tant, hi ha diferents mecàniques del joc que donen avantatge a l'equip que l'aconsegueixi o l'aprofiti millor. En conseqüència, aquestes mecàniques poden ser la clau per a guanyar una partida, tot i que hi ha moltes més variables a tenir en compte.

L'objectiu d'aquest notebook és analitzar el conjunt de dades o dataset de Partides Competitives (SOLO-Q) de *League of Legends* i veure si, amb les dades disponibles, es pot entrenar un model de predicció capaç de predir quin serà l'equip que guanyarà la partida. S'ha de tenir en compte que aquest dataset és de l'any 2019 i, per tant, pot no reflectir amb exactitud l'estat actual del joc i es tindrà en compte a les conclusions.

## 2. Explicació del *Gameplay*

Per a poder entendre i justificar les característiques del dataset, cal entendre algunes de les mecàniques d'una partida classificatòria de *League of Legends* (versió 2019).

### Objectiu principal

L'objectiu principal de la partida és el **Nexe** enemic, situat a la seva base: si el teu Nexe és destruït, perds la partida; si destrueixes el Nexe enemic, guanyes. El Nexe està protegit per dues Torretes interiors que cal destruir primer. Tota l'estratègia del joc gira al voltant dels objectius. Cada jugador de l'equip escull un campió diferent (personatge amb diferents atacs i habilitats).

### Línies

El mapa del joc està constituït per tres carrils, o línies (Top, Mid i Bot) i la jungla. Cada lí-

nia és defensada per les seves respectives estructures i l'objectiu és avançar per aquestes per tal d'arribar a la base enemiga. A part, cada línia va invocant minions, que són personatges no jugables (NPC). Els minons aliats ajuden a avançar per les línies, al mateix temps que donen experiència i or a l'equip enemig, mentre que els minions enemics fan el mateix de forma inversa.

## Estructures defensives

- **Torretes:** poden atacar als jugadors enemics si colpegen a l'enemic dins el seu radi. Protegeixen les línies i la base. Cada línia en té tres, tenint en compte la de l'inhibidor, i han de ser destruïdes en ordre per a cada línia. La primera Torreta destruïda atorga una bonificació d'or.
- **Inhibidors:** cada equip té tres Inhibidors, un per línia. Es troben a la base de l'equip i cada un és protegida per una Torreta que ha de ser destruïda. En destruir-ne un, l'equip passa a generar minions millorats només en aquella línia, facilitant l'avanç per ella. Com a mínim un inhibidor ha de ser destruït per accedir al Nexe.

## Monstres menors

Són com els minions, però per la jungla. Cal fer la distinció per no confondre amb els monstres següents.

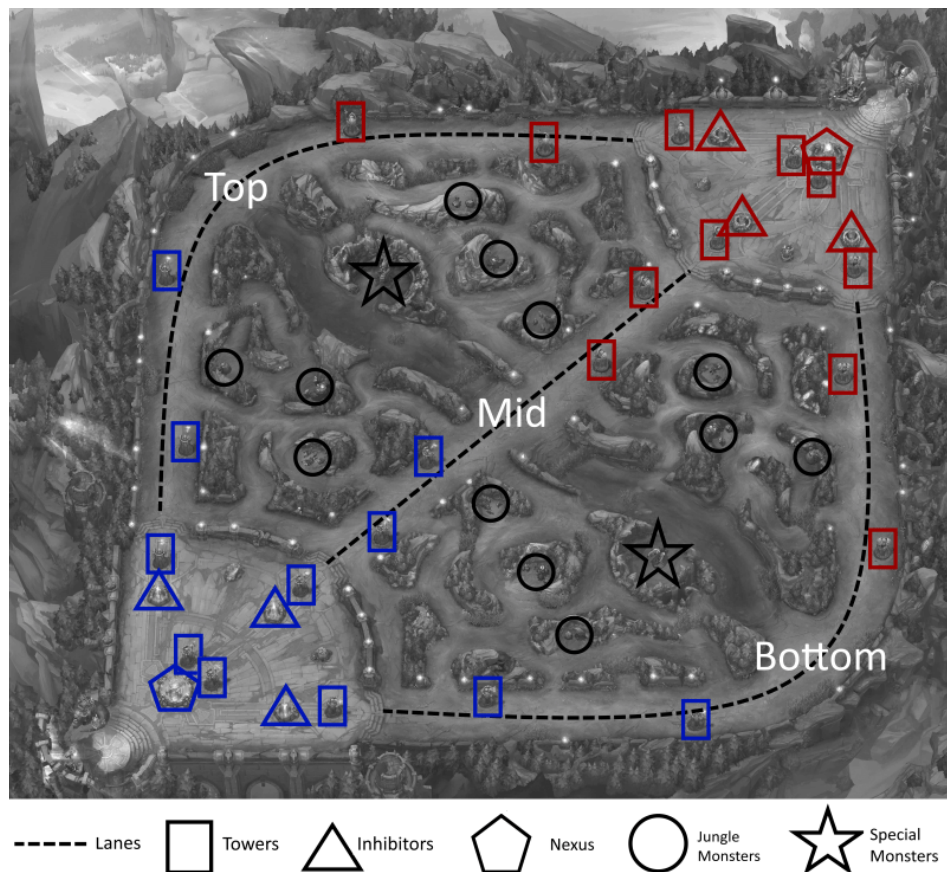
## Monstres èpics

- **Herald de l'escletxa:** apareix en el minut 10:00 de la partida. L'equip que el derroti pot aconseguir un objecte que invoca l'Herald, per ajudar a destruir les estructures enemigues d'una línia.
- **Dracs:** apareixen reiteradament durant el transcurs de la partida fins que un equip n'aconsegueixi quatre. Canvien d'element cada cop que apareixen i, depenent d'aquest, augmenta certes estadístiques de l'equip que l'ha aconseguit.
- **Drac ancià:** Apareix després de que un equip hagi matat a quatre dracs. Atorga més estadístiques diferents amb un efecte major sobre l'equip.
- **Baró Nashor:** apareix en el minut 20:00 de la partida. Atorga una millora temporal que incrementa la força de l'equip i dels minions, facilitant l'avanç per les línies i la base enemiga.

Amb diferents objectius, *farnejant* minions i monstres de la jungla i aconseguint morts enemigues (la primera mort atorga una bonificació d'or) l'equip obté or i experiència. També es pot col·locar un element de visió al mapa que s'anomena *Ward* o Guarda, en català. El *Ward* permet veure si un enemic es mou d'una línia a una altra o si s'estan fent un objectiu, és a dir, dona visió per a tenir un control sobre les decisions de l'equip rival. Es pot treure la visió de l'equip rival, i al fer-ho s'atorga una mica d'or. L'or permet la compra d'objectes que milloren les estadístiques dels campsions, mentre que l'experiència permet pujar de nivell les habilitats, facilitant l'obtenció de més objectius i, finalment, la

victòria. Cada cop que un jugador mor, reapareix en un temps determinat pel seu nivell. Per tant, a final de partida, trigaran més en reaparèixer.

En resum, la partida funciona de la següent manera: Els jugadors apareixen en la base en el minut 0:00, anant a les seves línies per a jugar. Durant el transcurs de la partida, intentaran avançar per aquestes, destruint les torretes, l'inhibidor i finalment, el Nexes i les seves torretes. També aniran matant als minions/monstres menors i campions enemics, alhora que intenten aconseguir els monstres èpic. Amb l'or, podran comprar objectes per enfortir-se i col·locar visió pel mapa per tenir un control més elevat en la partida.



Distribució del mapa de *League of Legends*

### 3. Dependències

En aquest apartat es troben totes les llibreries que s'han utilitzat en l'anàlisi del nostre dataset i, per tant, es requereixen per poder executar correctament el nostre notebook:

```
In [35]: #!pip install xgboost

from IPython.display import HTML
import matplotlib.pyplot as plt
import matplotlib.transforms as transforms
import numpy as np
import pandas as pd
import seaborn as sns

from sklearn.metrics import auc
from sklearn.base import clone
```

```

from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.svm import SVC
from sklearn.linear_model import RidgeClassifier, LogisticRegression
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.pipeline import Pipeline
from sklearn.pipeline import make_pipeline
from sklearn.compose import make_column_transformer
from sklearn.model_selection import GroupShuffleSplit
from sklearn.model_selection import GroupKFold
from sklearn.model_selection import cross_validate
from xgboost import XGBClassifier

```

## 4. Anàlisi exploratori del dataset

Per començar, és necessari analitzar i comprendre el dataset, tant les dades com el que representen. És un pas essencial per a poder identificar quin tipus d'informació tenim disponible i com pot influir en la construcció del nostre model de classificació (predir l'equip guanyador, variable objectiu "hasWon").

```

In [2]: df = pd.read_csv("../datasets/lol_ranked_games.csv")
pd.set_option("display.max_columns", None)
df

```

```

Out[2]:

```

|        | gameId     | gameDuration | hasWon | frame | goldDiff | expDiff | champLevelDiff |
|--------|------------|--------------|--------|-------|----------|---------|----------------|
| 0      | 4546233126 | 1443000      | 1      | 10    | -448     | -147    | -0.2           |
| 1      | 4546233126 | 1443000      | 1      | 12    | -1306    | -925    | -0.6           |
| 2      | 4546233126 | 1443000      | 1      | 14    | 2115     | 2578    | 0.4            |
| 3      | 4546233126 | 1443000      | 1      | 16    | 1195     | 2134    | 0.4            |
| 4      | 4546233126 | 1443000      | 1      | 18    | 2931     | 4382    | 0.6            |
| ...    | ...        | ...          | ...    | ...   | ...      | ...     | ...            |
| 242567 | 4402156483 | 1774000      | 0      | 30    | -8523    | -13498  | -1.6           |
| 242568 | 4379826739 | 1013000      | 0      | 10    | -271     | -1243   | -0.2           |
| 242569 | 4379826739 | 1013000      | 0      | 12    | -2013    | -3493   | -0.8           |
| 242570 | 4379826739 | 1013000      | 0      | 14    | -2388    | -4543   | -0.8           |
| 242571 | 4379826739 | 1013000      | 0      | 16    | -5601    | -7595   | -1.2           |

242572 rows × 59 columns

### 4.1 Informació bàsica del dataset

En aquesta secció mostrem els tipus de dades i la informació general del conjunt de dades.

```
In [3]: # Visualitzar informació bàsica del dataset
print("Data types i valors nuls:")

info_table = pd.DataFrame({
    'Data Type': df.dtypes,
    'Null Count': df.isna().sum()
})

from IPython.display import HTML, display

# Construïm el HTML amb un contenidor scrollable (vertical i horitzontal)
html = f"""
<div style="max-height:300px; overflow-y:auto; overflow-x:auto; border:1px solid
    {info_table.to_html(index=True)}
</div>
"""

display(HTML(html))

print("\nTotal de valors Nulls:", df.isna().sum().sum())
```

Data types i valors nuls:

|                       | Data Type | Null Count |
|-----------------------|-----------|------------|
| <b>gameId</b>         | int64     | 0          |
| <b>gameDuration</b>   | int64     | 0          |
| <b>hasWon</b>         | int64     | 0          |
| <b>frame</b>          | int64     | 0          |
| <b>goldDiff</b>       | int64     | 0          |
| <b>expDiff</b>        | int64     | 0          |
| <b>champLevelDiff</b> | float64   | 0          |
| <b>isFirstTower</b>   | int64     | 0          |
| <b>isFirstBlood</b>   | int64     | 0          |

Total de valors Nulls: 0

Explicació dels resultats:

#### Qualitat del Dataset:

- **242,572 registres complets:** suficient volum per entrenar models robustos
- **24,912 partides úniques:** diversitat de dades (no són repeticions de poques partides)
- **9.74 registres per partida:** significa que tenim aproximadament unes 9-10 snapshots temporals (*frames*) per partida
- **59 variables numèriques:** ric en features, sense valors faltants
- **Zero valors nuls:** dataset completament net (irreal a la realitat, haurà sigut imputat de manera prèvia)

#### Per al Model de Predicció:

- Tenim suficients dades per validació creuada i separació train/test (80/20)
- Cada partida té dades en múltiples *frames* temporals (no és només 1 snapshot per partida)
- Podem entrenar models sense preprocessament de dades faltants

**Tipus de dades:** Aquestes dades representen les estadístiques tenint en compte l'equip Blau com a objectiu. Totes les dades són numèriques, però mirant la primera taula podem veure com algunes variables representen el mateix concepte dos cops. Si un equip ha guanyat un objectiu (torreta, monstre epic, etc.) l'altre l'ha d'haver perdut. Però, realment representen el mateix? Anem a analitzar les dades. Podem organitzar les variables de la següent manera:

- Informació de la partida: informació general o tècnica de la partida. Variables: `gameId`, `gameDuration`, `Frame` i `hasWon` (equip Blau).
- Diferència entre equips: informació amb l'avantatge o desavantatge de l'equip Blau. Variables: `goldDiff`, `expDiff` i `champLevelDiff` (`expDiff` i `ChampLevelDiff` representen el mateix, ja que l'experiència és el que et fa pujar de nivell, per tant, s'haurà de determinar si cal avaluar les dues).
- El primer en fer algun objectiu: si l'equip Blau ha sigut el primer equip que ha aconseguit aquell objectiu en específic. Variables: `isFirstTower` i `isFirstBlood`.
- Matar objectius: informació sobre quins objectius (monstres) ha aconseguit l'equip Blau. Variables: `killedFireDrake`, `killedWaterDrake`, `killedAirDrake`, `killedEarthDrake`, `killedElderDrake`, `killedBaronNashor` i `killedRiftHerald`.
- Objectius perduts (monstres): informació amb els objectius (monstres) que ha aconseguit l'equip Vermell (o objectius que ha perdut l'equip Blau). Variables: `lostFireDrake`, `lostWaterDrake`, `lostAirDrake`, `lostEarthDrake`, `lostElderDrake`, `lostBaronNashor` i `lostRiftHerald`.

**ACLARACIÓ:** odem concloure que **NO** representen el mateix, un equip pot no fer-se un objectiu ja sigui perquè l'equip enemic l'ha aconseguit, perquè no hagi aparegut o perquè cap equip l'ha aconseguit. Per tant, s'han de tenir en compte totes dues ja que només amb una no es pot determinar l'altra amb exactitud.

- Objectius destruïts: informació amb els objectius (estructures) que ha aconseguit l'equip Blau:

Exemple de variables: `destroyedTop/Mid/BotInhibitor`, `destroyedTop/Mid/BotNexusTurret`, `destroyedBotBaseTurret`, etc.

- Objectius perduts (estructures):

Exemple de variables: `lostTop/Mid/BotInhibitor`, `lostTop/Mid/BotNexusTurret`, `lostBotBaseTurret`, etc.

**IMPORTANT:** no hi ha torretes a la base de totes 3 línies, existeixen les dades

(**destroyedTopBaseTurret** i **destroyedMidBaseTurret**) però només existeixen dues torretes per protegir el Nexa, per això "**destroyedBotBaseTurret**" pot agafar el valor de dos. Per tant, les altres dues entrades poden ser eliminades ja que el valor és sempre zero a totes les 242572 entrades. Caldrà renombrar les columnes per a que sigui una representació fidel del joc.

- Estadístiques de KDA (*Kills/Deaths/Assists*): estadístiques sobre els assassinats a l'equip enemic i morts i assistències de l'equip Blau.

Variables: *kills*, *deaths* i *assists*.

- Estadístiques de visió: estadístiques sobre la visió (*Wards*) de l'equip Blau:

Variables: *wardsPlaced*, *wardsDestroyed* i *wardsLost*

## 4.2 Estadístiques descriptives

En aquesta secció mostrem les estadístiques descriptives del dataset (mitjana, mediana, desviació estàndard, etc.)

```
In [4]: desc_table = df.describe()

html = f"""
<div style="max-height:300px; overflow-y:auto; overflow-x:auto;
        border:1px solid #ccc; padding:6px;">
    {desc_table.to_html()}
</div>
"""

display(HTML(html))
```

|              | gameId       | gameDuration | hasWon        | frame         | goldDiff      |    |
|--------------|--------------|--------------|---------------|---------------|---------------|----|
| <b>count</b> | 2.425720e+05 | 2.425720e+05 | 242572.000000 | 242572.000000 | 242572.000000 | 24 |
| <b>mean</b>  | 4.501480e+09 | 1.785494e+06 | 0.498128      | 19.811264     | -11.413811    |    |
| <b>std</b>   | 5.331693e+07 | 3.761901e+05 | 0.499998      | 7.197339      | 5438.051920   |    |
| <b>min</b>   | 4.357970e+09 | 5.490000e+05 | 0.000000      | 10.000000     | -21578.000000 | -3 |
| <b>25%</b>   | 4.463051e+09 | 1.532000e+06 | 0.000000      | 14.000000     | -3420.250000  | -  |
| <b>50%</b>   | 4.529395e+09 | 1.774000e+06 | 0.000000      | 18.000000     | 24.000000     |    |
| <b>75%</b>   | 4.543718e+09 | 2.025000e+06 | 1.000000      | 24.000000     | 3409.000000   |    |
| <b>max</b>   | 4.547671e+09 | 3.428000e+06 | 1.000000      | 56.000000     | 23432.000000  | 4  |

Explicació dels resultats:

### 1. Diferència d'Or (goldDiff - CRÍTICA):

- Rang de -14,000 a +14,000 or mostra una dominació en aquest aspecte sobre

l'altre equip.

- Desviació estàndard de 4,700 indica diferències significatives entre equips.

## 2. **Frames Temporals:**

- Varia de *frame* 10 a 56 (representació dels minuts 10 a 56 en intervals de 2: min=10, 12, 14, ..., max=56).
- La majoria de partides es troben entre els *frames* 10-30, per tant l'inici de la partida probablement serà més determinant.

## 3. **Problemes:**

- Com hem determinat abans, els atributs "destroyedTopBaseTurret" i "destroyedMidBaseTurret" no serveixen per res, no aporten cap valor al dataset. Per tant, com s'anomenen certes columnes no representen el que passa dins del joc. Tenim "destroyedBotBaseTurret" que representa les dues torretes que defensen al Nexe, i per tant "destroyedTopBaseTurret" i "destroyedMidBaseTurret" no representen res, al igual que els seves variacions "lost". Però tenim unes altres variables anomenades "destroyedTopNexusTurret", "destroyedMidNexusTurret" i "destroyedBotNexusTurret", que són en realitat les torretes que defensen als inhibidors. Per tant els noms estan intercanviats i sobren un parell de columnes. Això ho podem canviar a continuació.
- La visió colocada (wardsPlaced) té un màxim de 1014, la qual cosa és impossible. Amb una espera d'uns 90 segons per col·locar un, amb 5 jugadors (tenint en compte que el support pot col·locar 3 a partir de cert punt), el màxim hauria d'estar entre 220 i 400 (sent 400 un valor per tal de no eliminar possibles valors reals), per tant hi ha algun problema amb aquest atribut. Analitzarem aquest problema amb més profunditat en l'anàlisi de la visió.

```
In [5]: # Eliminar i renombrar columnes per representar millor les dades
cols_to_drop = [
    "destroyedTopBaseTurret",
    "destroyedMidBaseTurret",
    "lostTopBaseTurret",
    "lostMidBaseTurret"
]

df = df.drop(columns=cols_to_drop)

df = df.rename(columns={
    # Torres del Nexus són equivalents a Torres dels inhibidors
    "destroyedTopNexusTurret": "destroyedTopInhibitorTurret",
    "destroyedMidNexusTurret": "destroyedMidInhibitorTurret",
    "destroyedBotNexusTurret": "destroyedBotInhibitorTurret",
    "lostTopNexusTurret": "lostTopInhibitorTurret",
    "lostMidNexusTurret": "lostMidInhibitorTurret",
    "lostBotNexusTurret": "lostBotInhibitorTurret",
    # Torres de la base són equivalents a Torres del Nexus
    "destroyedBotBaseTurret": "destroyedNexusTurrets",
    "lostBotBaseTurret": "lostNexusTurrets"
})
```

## 4.3 Distribució de la variable objectiu

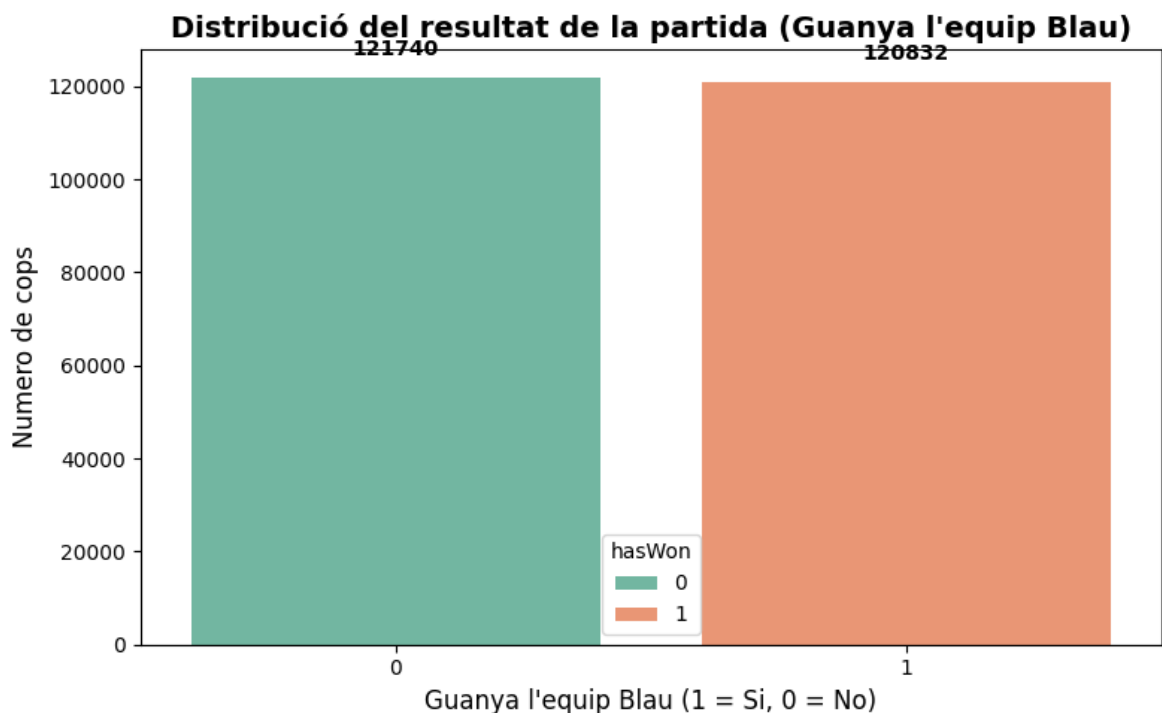
En aquesta secció analitzem la distribució de la variable objectiu (hasWon) que indica quin equip ha guanyat.

```
In [6]: # Distribució de la variable objectiu
print(df["hasWon"].value_counts())
print("\nPercentatge de la distribució:")
print(df["hasWon"].value_counts(normalize=True) * 100)

# Visualització de la distribució
plt.figure(figsize=(8, 5))
sns.countplot(data=df, x="hasWon", hue="hasWon", palette="Set2")
plt.title("Distribució del resultat de la partida (Guanya l'equip Blau)", fontsi
plt.xlabel("Guanya l'equip Blau (1 = Si, 0 = No)", fontsize=12)
plt.ylabel("Numero de cops", fontsize=12)
for i, v in enumerate(df["hasWon"].value_counts()):
    plt.text(i, v + 5000, str(v), ha="center", fontweight='bold')
plt.tight_layout()
plt.show()
```

```
hasWon
0    121740
1    120832
Name: count, dtype: int64
```

```
Percentatge de la distribució:
hasWon
0     50.187161
1     49.812839
Name: proportion, dtype: float64
```



- **Partides Perdudes (hasWon=0):** 121,740 (50.19%).
- **Partides Guanyades (hasWon=1):** 120,832 (49.81%).
- **Diferència:** Quasi perfectament equilibrat (0.38% de diferència).

Amb aquests resultats, podem determinar el següent:

### 1. Dataset equilibrat:

- No hi ha biaix cap a una classe (no hi ha més victòries que derrotes).
- Percentatges 50-50 són ideals per entrenar classificadors.
- No necessitem tècniques de balanceig (SMOTE, class weights, etc.).

### 2. Implicacions per al Model:

- **Baseline accuracy:** 50% (predicció aleatòria donarà 50% d'exactitud).
- **Target per millorar:** El nostre model hauria d'assolir un número superior a aquest 50%.
- **No hi ha overfitting:** Dataset equilibrat.

## 4.4 Correlació entre variables i la variable objectiu

Analitzem la correlació entre les variables numèriques i la variable objectiu per tal de determinar quines dades són les que ens ajudaran a predir quin equip guanyarà.

```
In [7]: # Correlació de variables amb la variable objectiu
df2 = df.copy()
correlation_with_target = df2.corr()["hasWon"].sort_values(ascending=False)

# Visualització de la correlació amb la variable objectiu
plt.figure(figsize=(12, 10))
plt.barh(range(len(correlation_with_target)), correlation_with_target.values)
plt.yticks(range(len(correlation_with_target)), correlation_with_target.index)
plt.xlabel("Correlació amb 'hasWon'", fontsize=12)
plt.title("Correlació de les característiques amb la variable objectiu", fontsize=12)
plt.axvline(x=0, color="r", linestyle="--", alpha=0.3)
plt.tight_layout()
plt.show()

winner_correlation = df.corr()["hasWon"].to_frame().T

fig, ax = plt.subplots(figsize=(15, 4))

sns.heatmap(
    winner_correlation,
    cmap='RdBu_r',
    center=0,
    vmin=-1, vmax=1,
    square=True,
    linewidths=0.5,
    cbar_kws={"shrink": 0.6},
    ax=ax
)

# --- Move entire x-axis downward (important!) ---
ax.xaxis.set_ticks_position('bottom')
ax.xaxis.set_label_position('bottom')

# Rotació
ax.set_xticklabels(ax.get_xticklabels(), rotation=90, ha='center')

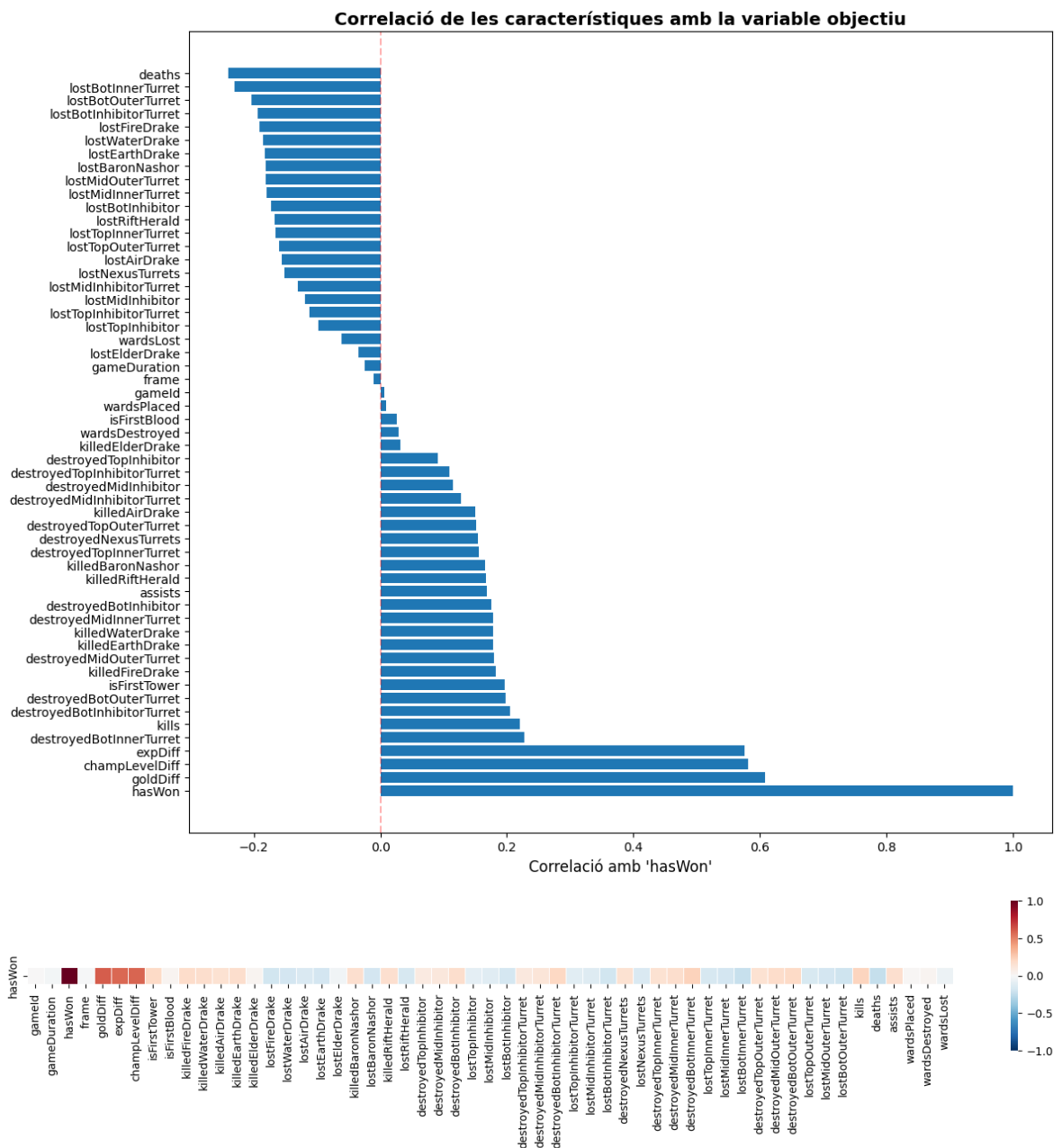
# --- Alternar etiquetes sense que entrin al heatmap ---
for i, label in enumerate(ax.get_xticklabels()):
```

```

if i % 2 == 0:
    label.set_y(-0.08) # etiqueta una mica més avall
else:
    label.set_y(-0.18) # etiqueta encara més avall

plt.tight_layout()
plt.show()

```



Les variables mostrades tenen relacions lineals molt dèbils entre elles, no hi ha patrons predictius clars entre aquestes característiques. De totes maneres, algunes columnes si que tenen una correlació més alta, algunes de forma positiva i unes altres de forma negativa. Volem veure quins són els atributs que tenen una correlació més directa amb el que volem predir:

```

In [8]: correlation_threshold = 0.19 # Ajustat per incloure més atributs, no només el pa
        useful = {
            col: correlation_with_target[col]
            for col in correlation_with_target.index
            if abs(float(correlation_with_target[col])) > correlation_threshold
        }

```

```
# Ordenem per força de correlació
sorted_useful = sorted(useful.items(), key=lambda x: abs(x[1]), reverse=True)

useful_columns = [col for col, corr in sorted_useful]

print("Total d'atributs utils:", len(sorted_useful)-1)
print("Atributs utils (ordenats):")
for col, corr in sorted_useful:
    if col != "hasWon":
        print(f"{col:30s} corr = {corr:.4f} \n")
```

```
Total d'atributs utils: 13
Atributs utils (ordenats):
goldDiff                corr = 0.6075

champLevelDiff          corr = 0.5810

expDiff                 corr = 0.5755

deaths                  corr = -0.2403

lostBotInnerTurret      corr = -0.2299

destroyedBotInnerTurret corr = 0.2277

kills                   corr = 0.2198

destroyedBotInhibitorTurret corr = 0.2049

lostBotOuterTurret      corr = -0.2034

destroyedBotOuterTurret corr = 0.1977

isFirstTower            corr = 0.1962

lostBotInhibitorTurret  corr = -0.1945

lostFireDrake           corr = -0.1914
```

### Predictors més importants:

1. **goldDiff**: La diferència d'or és el millor predictor de victòria.
  - Equips guanyadors dominen econòmicament els equips perdedors.
2. **champLevelDiff**: La diferència d'experiència.
  - Si vas per sobre en experiència, vas per sobre també en nivell, és la conseqüència de 'expDiff'.
3. **expDiff**: La diferència d'experiència és quasi tan important com l'or.
  - Indica control general del joc i *kills/farm*.
4. **destroyedBotInnerTurret**, **destroyedBotInhibitorTurret**, **destroyedBotOuterTurret**: Destruir les estructures de la línia inferior són les que més afecten al resultat:

5. ***kills***: Més *kills* = més probabilitat de guanyar.

6. **isFirstTower**: Destruir la primera estructura de tota la partida aporta un avantatge.

#### Correlacions negatives importants:

- ***deaths***: Més morts = menys probabilitat de guanyar.
- **lostBotInnerTurret, lostBotOuterTurret, lostBotInhibitorTurret**: La destrucció de les estructures de la línia inferior afecta més que perdre les de les altres línies.
- **lostFireDrake**: Els dracs de foc són l'objectiu que afecta més al resultat de la partida, augmentant les probabilitats de perdre.

Per tant, les estructures de la línia inferior, *kills/deaths* i la diferencia d'estadístiques entre equips són les característiques que més s'han de tenir en compte a l'hora de fer el nostre model. L'única estadística a recalcar dels monstres com a objectiu, és el drac de foc. Per tant, analitzarem més detalladament aquestes característiques a continuació.

## 4.5 Anàlisi per trams temporals (*frames*)

Analitzem com els predictors més importants canvien al llarg del joc, per fer això analitzem les estadístiques en els diferents *frames*.

```
In [9]: # Anàlisis de la distribució temporal
print("Partides uniques totals:", {df['gameId'].nunique()})
print("Registres totals (frames):", {len(df)})
print(f"Mitjana de frames per partida: {len(df) / df['gameId'].nunique():.2f}")

def show_scrollable(obj, max_height=300):
    """Mostra Series o DataFrames dins d'un div scrollable."""
    # Converteix a DataFrame per garantir format taula
    if isinstance(obj, pd.Series):
        obj = obj.to_frame()

    html = f"""
    <div style="max-height:{max_height}px; overflow-y:auto; overflow-x:auto;
        border:1px solid #ccc; padding:6px;">
        {obj.to_html()}
    </div>
    """
    display(HTML(html))

print("\nEstadístiques de durada de la partida (milisegons):")
show_scrollable(df['gameDuration'].describe())

print("Distribució de frames:")
show_scrollable(df['frame'].value_counts().sort_index())

# Visualització de frames
plt.figure(figsize=(12, 5))
frame_counts = df['frame'].value_counts().sort_index()
plt.bar(frame_counts.index, frame_counts.values, color='steelblue')
```

```
plt.xlabel("Frames de la partida (10-min intervals)", fontsize=12)
plt.ylabel("Nombre de registres", fontsize=12)
plt.title("Distribució de registres per Frame", fontsize=14, fontweight='bold')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```

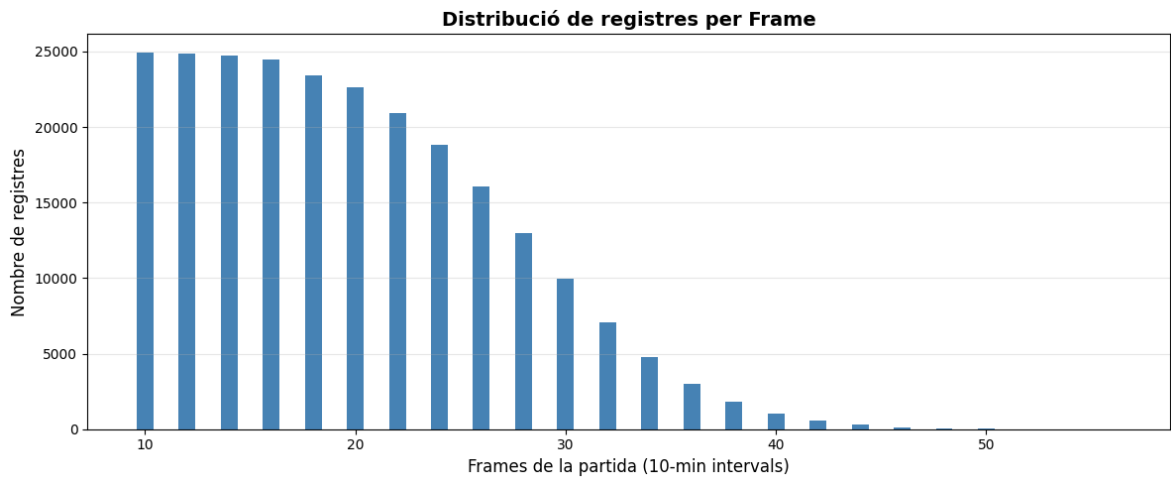
Partides uniques totals: {24912}  
Registres totals (frames): {242572}  
Mitjana de frames per partida: 9.74

Estadístiques de durada de la partida (milisegons):

| gameDuration |              |
|--------------|--------------|
| count        | 2.425720e+05 |
| mean         | 1.785494e+06 |
| std          | 3.761901e+05 |
| min          | 5.490000e+05 |
| 25%          | 1.532000e+06 |
| 50%          | 1.774000e+06 |
| 75%          | 2.025000e+06 |
| max          | 3.428000e+06 |

Distribució de frames:

| count |       |
|-------|-------|
| frame |       |
| 10    | 24912 |
| 12    | 24842 |
| 14    | 24716 |
| 16    | 24502 |
| 18    | 23430 |
| 20    | 22632 |
| 22    | 20951 |
| 24    | 18818 |



### Estructura del dataset:

- **24,912 partides úniques** amb una mitjana de 9.74 registres per partida.
- **Frames temporals:** Des del frame 10 (minut 10) fins al frame 56 (minut 56).
- **Dropout natural:** Només 1,013 partides arriben al frame 40, la resta es decideixen abans.

### Implicacions:

1. **Duració mitjana:** aproximadament 30 minuts per partida (1,785,494 ms ≈ 29.7min).
2. **Distribució de frames:** La majoria de partides acaben entre frame 20-26 (minuts 20-26).
3. **Decisivitat:** La meitat de partides estan decidides al frame 22 (minut 22).

Podem determinar que els frames inicials (10-14) són importants per establir dominància i garantir la victòria. I tampoc és comú acabar abans d'aquest temps. Tenint en compte aquesta distribució, anem a veure com les estadístiques evolucionen durant el transcurs d'una partida.

## 4.6 Evolució de les estadístiques de joc per frame

```
In [10]: # Anàlisis de l'evolució per frame de les estadístiques clau per les partides guanyades i perdudes
# Agrupar per frame i calcular la mitjana per a les partides guanyades i perdudes
frame_stats = df.groupby(['frame', 'hasWon']).agg({
    'goldDiff': 'mean',
    'expDiff': 'mean',
    'champLevelDiff': 'mean',
    'kills': 'mean',
    'deaths': 'mean'
}).reset_index()

print("Mitjana de les estadístiques per frame i status guanyador:")

def show_scrollable(obj, max_height=300):
    """Mostra Series o DataFrames dins d'un div scrollable."""
    if isinstance(obj, pd.Series):
        obj = obj.to_frame()
    html = f"""
```

```

<div style="max-height:{max_height}px; overflow-y:auto; overflow-x:auto;
        border:1px solid #ccc; padding:6px;">
    {obj.to_html(index=False)}
</div>
"""
display(HTML(html))

show_scrollable(frame_stats.head(20))

# Visualització de l'evolució per frames
fig, axes = plt.subplots(2, 2, figsize=(15, 10))

# Diferència d'evolució de l'or
ax = axes[0, 0]
for won_status in [0, 1]:
    data = frame_stats[frame_stats['hasWon'] == won_status]
    label = 'Equip Blau Guanya' if won_status == 1 else 'Equip Blau Perd'
    ax.plot(data['frame'], data['goldDiff'], marker='o', label=label, linewidth=2)
ax.set_xlabel('Frame de la partida', fontsize=11)
ax.set_ylabel("Diferència d'or promig", fontsize=11)
ax.set_title("Evolució de la diferència d'or per Frame", fontsize=12, fontweight='bold')
ax.legend()
ax.grid(True, alpha=0.3)

# Diferència d'evolució de l'experiència
ax = axes[0, 1]
for won_status in [0, 1]:
    data = frame_stats[frame_stats['hasWon'] == won_status]
    label = 'Equip Blau Guanya' if won_status == 1 else 'Equip Blau Perd'
    ax.plot(data['frame'], data['expDiff'], marker='o', label=label, linewidth=2)
ax.set_xlabel('Frame de la partida', fontsize=11)
ax.set_ylabel("Diferència d'experiència promig", fontsize=11)
ax.set_title("Evolució de la diferència d'experiència per frame", fontsize=12, fontweight='bold')
ax.legend()
ax.grid(True, alpha=0.3)

# Diferència d'evolució del nivell de personatge
ax = axes[1, 0]
for won_status in [0, 1]:
    data = frame_stats[frame_stats['hasWon'] == won_status]
    label = 'Equip Blau Guanya' if won_status == 1 else 'Equip Blau Perd'
    ax.plot(data['frame'], data['champLevelDiff'], marker='o', label=label, linewidth=2)
ax.set_xlabel('Frame de la partida', fontsize=11)
ax.set_ylabel("Diferència del nivell de personatge promig", fontsize=11)
ax.set_title("Evolució de la diferència del nivell de personatge per frame", fontsize=12, fontweight='bold')
ax.legend()
ax.grid(True, alpha=0.3)

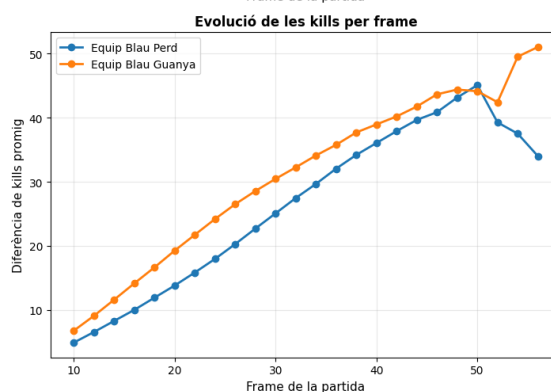
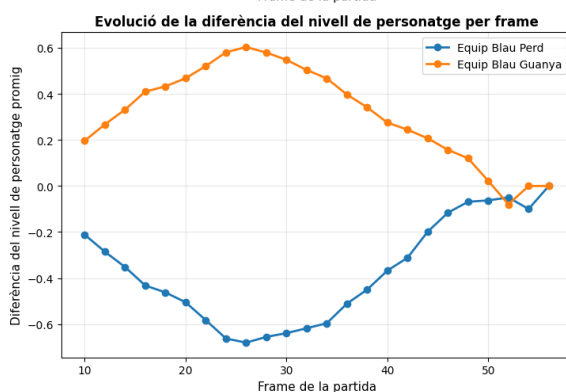
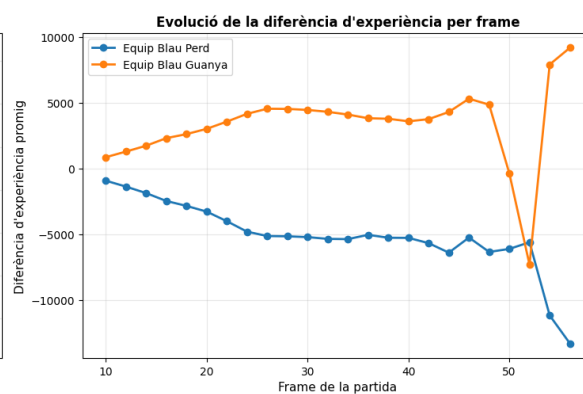
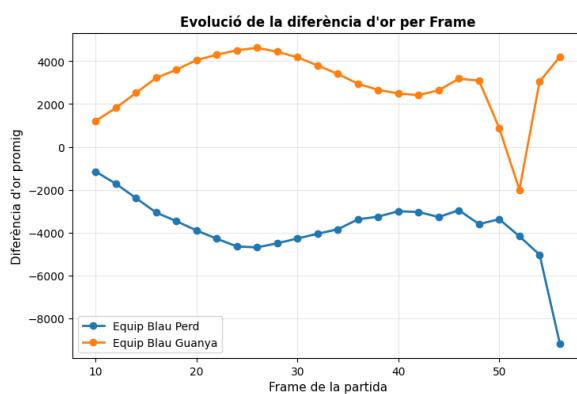
# Evolució de les kills
ax = axes[1, 1]
for won_status in [0, 1]:
    data = frame_stats[frame_stats['hasWon'] == won_status]
    label = 'Equip Blau Guanya' if won_status == 1 else 'Equip Blau Perd'
    ax.plot(data['frame'], data['kills'], marker='o', label=label, linewidth=2)
ax.set_xlabel('Frame de la partida', fontsize=11)
ax.set_ylabel("Diferència de kills promig", fontsize=11)
ax.set_title('Evolució de les kills per frame', fontsize=12, fontweight='bold')
ax.legend()
ax.grid(True, alpha=0.3)

```

```
plt.tight_layout()
plt.show()
```

Mitjana de les estadístiques per frame i status guanyador:

| frame | hasWon | goldDiff     | expDiff      | champLevelDiff | kills     | deaths    |
|-------|--------|--------------|--------------|----------------|-----------|-----------|
| 10    | 0      | -1153.235109 | -923.141195  | -0.211574      | 4.922493  | 6.711226  |
| 10    | 1      | 1215.184066  | 859.972247   | 0.196156       | 6.781243  | 4.841614  |
| 12    | 0      | -1722.137384 | -1374.511948 | -0.285330      | 6.573026  | 9.034994  |
| 12    | 1      | 1816.679363  | 1288.133632  | 0.267296       | 9.100744  | 6.487797  |
| 14    | 0      | -2392.247742 | -1868.422016 | -0.351818      | 8.301684  | 11.501749 |
| 14    | 1      | 2518.439920  | 1733.852394  | 0.331332       | 11.609980 | 8.217867  |
| 16    | 0      | -3069.257609 | -2468.302814 | -0.432012      | 10.025515 | 14.028222 |
| 16    | 1      | 3219.743767  | 2310.111102  | 0.409453       | 14.158532 | 9.922277  |
| 18    | 0      | -3469.556060 | -2842.446938 | -0.462116      | 11.928308 | 16.524882 |



### Fase inicial (Frame 10-14):

- Diferència d'or inicial:  $\pm 1,200$ -2,400 entre equips.
- Diferència de *kills*: Els guanyadors ja porten 6-8 *kills*, perdedors 4-5.
- Els equips que dominen *early game* mantenen l'avantatge quasi tota la partida.

### Fase mitja (Frame 16-26):

- **Màxim de diferència d'or**: aproximadament 4,500 d'or al frame 26.
- **Diferència de *kills***: Els guanyadors acumulen 26+ *kills*, perdedors 20+.

### Fase final (Frame 28+):

- Les diferències comencen a estabilitzar-se.
- El finals està decidit per una bona jugada (una bona batalla es definitiva)
- Com hi ha un màxim de nivell per personatge (nivell 18 per a tothom), aquesta estadística aconsegueix equilibrar-se.

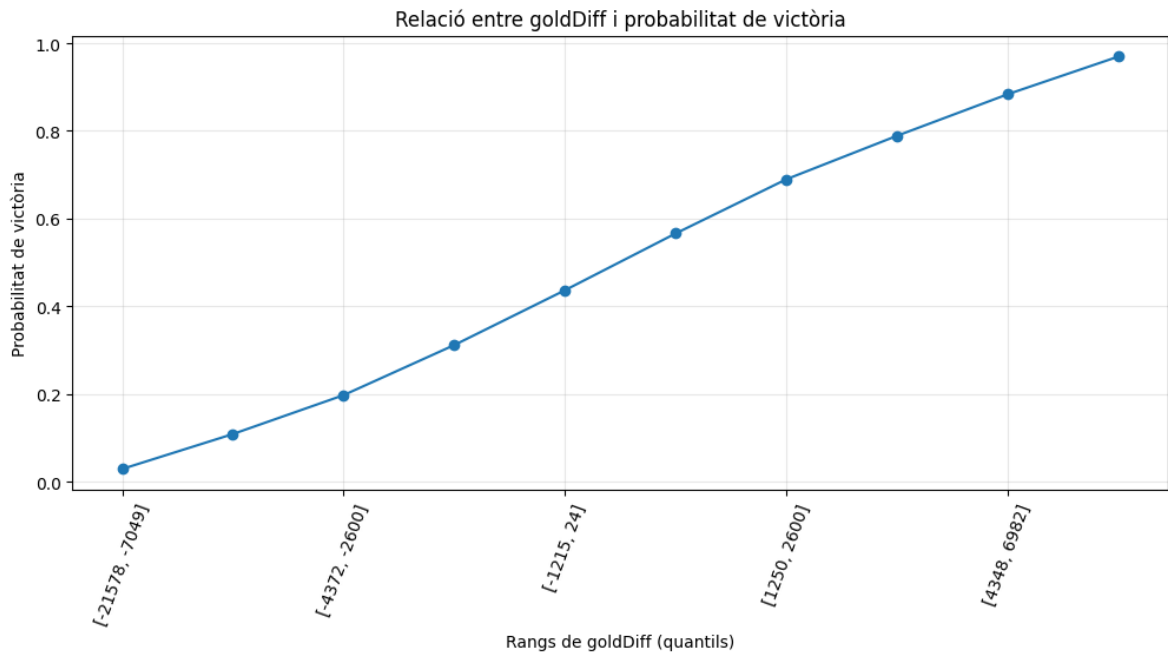
Ara bé, anem a veure com l'or determina la victòria:

A mesura que avança la partida, la diferència d'or i experiència es fa tan gran que el model té menys utilitat predictiva, perquè la victòria es fa òbvia (tot i que no és garantida).

```
In [11]: df['goldDiff_quantile'] = pd.qcut(df['goldDiff'], q=10, labels=False, duplicates
prob = df.groupby('goldDiff_quantile')['hasWon'].mean()

# Obtenir els rangs reals de cada quantil
quantile_ranges = df.groupby('goldDiff_quantile')['goldDiff'].agg(['min', 'max'])
prob.index = [f'[{int(row["min"])}, {int(row["max"])}]'
              for _, row in quantile_ranges.iterrows()]

prob.plot(figsize=(12,5), marker='o')
plt.xticks(rotation=70)
plt.ylabel("Probabilitat de victòria")
plt.xlabel("Rangs de goldDiff (quantils)")
plt.title("Relació entre goldDiff i probabilitat de victòria")
plt.grid(alpha=0.3)
plt.show()
```



A mesura que avança la partida, la diferència d'or i experiència es fa tan gran que el model té menys utilitat predictiva, perquè la victòria es fa òbvia (tot i que no és garantida).

A mesura que la diferència d'or augmenta, la probabilitat de victòria creix de manera gairebé lineal fins a arribar a 100% quan l'avantatge supera aproximadament una quantitat de 4500 d'or.

## 4.7 Anàlisi d'objectius èpics (Dracs, Baron Nashor, Rift Herald)

Analitzem la importància de la captura dels monstres èpics en la victòria.

```
In [12]: # Dracs aconseguits i perduts
dragons_data = df.groupby('hasWon').agg({
    'killedFireDrake': 'mean',
    'killedWaterDrake': 'mean',
    'killedAirDrake': 'mean',
    'killedEarthDrake': 'mean',
    'killedElderDrake': 'mean',
    'lostFireDrake': 'mean',
    'lostWaterDrake': 'mean',
    'lostAirDrake': 'mean',
    'lostEarthDrake': 'mean',
    'lostElderDrake': 'mean',
    'killedBaronNashor': 'mean',
    'lostBaronNashor': 'mean',
    'killedRiftHerald': 'mean',
    'lostRiftHerald': 'mean',
}).round(4)

print("Mitjanes dels objectius segons l'estat de hasWon:")

html = f"""
<div style="max-height:300px; overflow-y:auto; overflow-x:auto;
        border:1px solid #ccc; padding:6px;">
    {dragons_data.to_html()}
</div>
"""

display(HTML(html))

# Visualització dels objectius
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Dracs
ax = axes[0]
dragon_types = ['Fire', 'Water', 'Air', 'Earth', 'Elder']
wins = [df[df['hasWon']==1]['killedFireDrake'].mean(),
        df[df['hasWon']==1]['killedWaterDrake'].mean(),
        df[df['hasWon']==1]['killedAirDrake'].mean(),
        df[df['hasWon']==1]['killedEarthDrake'].mean(),
        df[df['hasWon']==1]['killedElderDrake'].mean()]
losses = [df[df['hasWon']==0]['killedFireDrake'].mean(),
          df[df['hasWon']==0]['killedWaterDrake'].mean(),
          df[df['hasWon']==0]['killedAirDrake'].mean(),
```

```

df[df['hasWon']==0]['killedEarthDrake'].mean(),
df[df['hasWon']==0]['killedElderDrake'].mean())

x = np.arange(len(dragon_types))
width = 0.35
ax.bar(x - width/2, wins, width, label='Equip Blau Guanya', color='steelblue')
ax.bar(x + width/2, losses, width, label='Equip Blau Perd', color='coral')
ax.set_xlabel('Dragon Type', fontsize=11)
ax.set_ylabel('Average Dragon Kills', fontsize=11)
ax.set_title('Dragons Killed by Outcome', fontsize=12, fontweight='bold')
ax.set_xticks(x)
ax.set_xticklabels(dragon_types, rotation=45)
ax.legend()
ax.grid(axis='y', alpha=0.3)

# Baron and Rift Herald
ax = axes[1]
objectives = ['Baron', 'Rift Herald']
wins_obj = [df[df['hasWon']==1]['killedBaronNashor'].mean(),
            df[df['hasWon']==1]['killedRiftHerald'].mean()]
losses_obj = [df[df['hasWon']==0]['killedBaronNashor'].mean(),
             df[df['hasWon']==0]['killedRiftHerald'].mean()]

x = np.arange(len(objectives))
ax.bar(x - width/2, wins_obj, width, label='Blue Team Wins', color='steelblue')
ax.bar(x + width/2, losses_obj, width, label='Blue Team Loses', color='coral')
ax.set_xlabel('Objective Type', fontsize=11)
ax.set_ylabel('Average Dragon Kills', fontsize=11)
ax.set_title('Baron & Rift Herald Killed by Outcome', fontsize=12, fontweight='b')
ax.set_xticks(x)
ax.set_xticklabels(objectives)
ax.legend()
ax.grid(axis='y', alpha=0.3)

# Primers objectius
ax = axes[2]
first_objs = ['First Tower', 'First Blood']
wins_first = [df[df['hasWon']==1]['isFirstTower'].mean() * 100,
              df[df['hasWon']==1]['isFirstBlood'].mean() * 100]
losses_first = [df[df['hasWon']==0]['isFirstTower'].mean() * 100,
                df[df['hasWon']==0]['isFirstBlood'].mean() * 100]

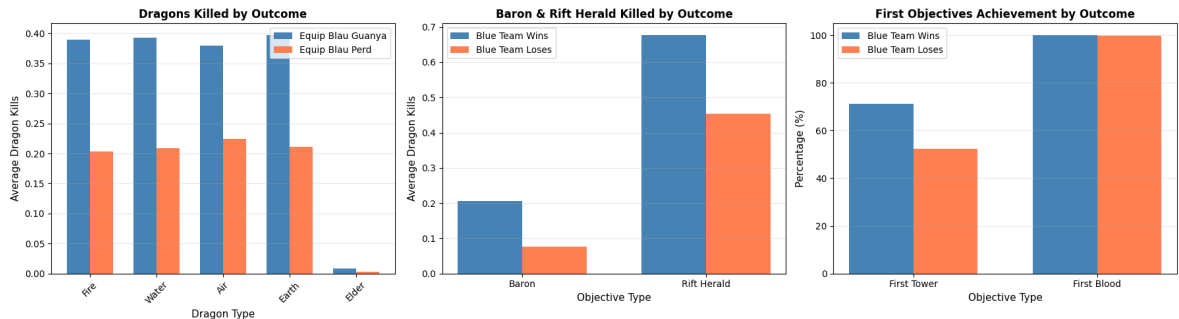
x = np.arange(len(first_objs))
ax.bar(x - width/2, wins_first, width, label='Blue Team Wins', color='steelblue')
ax.bar(x + width/2, losses_first, width, label='Blue Team Loses', color='coral')
ax.set_xlabel('Objective Type', fontsize=11)
ax.set_ylabel('Percentage (%)', fontsize=11)
ax.set_title('First Objectives Achievement by Outcome', fontsize=12, fontweight='b')
ax.set_xticks(x)
ax.set_xticklabels(first_objs)
ax.legend()
ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()

```

Mitjanes dels objectius segons l'estat de hasWon:

|        | killedFireDrake | killedWaterDrake | killedAirDrake | killedEarthDrake | killedElderDragon |
|--------|-----------------|------------------|----------------|------------------|-------------------|
| hasWon |                 |                  |                |                  |                   |
| 0      | 0.2035          | 0.2086           | 0.2246         | 0.2111           |                   |
| 1      | 0.3890          | 0.3926           | 0.3799         | 0.3968           |                   |



## Dracs:

### • Guanyadors vs. Perdedors:

- Fire Drake: 0.39 kills vs. 0.20 (95% més per guanyadors).
- Water Drake: 0.39 vs. 0.21 (86% més).
- Air Drake: 0.38 vs. 0.22 (73% més).
- Earth Drake: 0.40 vs. 0.21 (90% més).

Cada drac capturat augmenta significativament les probabilitats de guanyar. Els equips predestinats a guanyar són els que tenen un major control sobre els dracs. Els dracs tenen un impacte acumulatiu molt més important que qualsevol altre objectiu individual: són el millor predictor d'avantatge escalat.

## Baron Nashor:

- **Guanyadors:** promig de 0.21 kills.
- **Perdedors:** promig de 0.08 kills.
- Un 260% més per als guanyadors. Aquest és l'objectiu més decisiu. El problema és que la partida es sol decidir abans de la seva aparició. Els equips que aconseguixen el Baron Nashor rarament perden la partida després

## Rift Herald:

- **Guanyadors:** 0.68 kills (67.6% de les partides)
- **Perdedors:** 0.45 kills (45.3% de les partides) És més accessible que el Baron, però afecta menys al resultat de la partida.

## Primers objectius:

- **First Tower:** Guanyadors 72%, Perdedors 28%. Per tant fer-se la primera Torreta té un efecte sobre la partida.
- **First Blood:** Guanyadors 50%, Perdedors 50%. No té cap rellevància, tenim un 50% de possibilitats d'encertar el resultat amb aquesta característica, no té efecte real so-

bre la partida.

Podem concloure que el control d'objectius (monstres) és millor predictor de victòria que les estadístiques de combat, però només si s'arriba al moment de disputar aquests objectius.

## 4.8 Anàlisi de destrucció d'estructures

Analitzem la relació entre la destrucció d'estructures defensives i la victòria.

```
In [13]: eda_df = df.copy()

# Estadístiques de torres i inhibidors segons hasWon
tower_inhibitor_stats = eda_df.groupby('hasWon').agg({
    'destroyedTopInhibitor': 'mean',
    'destroyedMidInhibitor': 'mean',
    'destroyedBotInhibitor': 'mean',
    'destroyedTopInhibitorTurret': 'mean',
    'destroyedMidInhibitorTurret': 'mean',
    'destroyedBotInhibitorTurret': 'mean',
    'destroyedNexusTurrets': 'mean',
    'destroyedTopInnerTurret': 'mean',
    'destroyedMidInnerTurret': 'mean',
    'destroyedBotInnerTurret': 'mean',
    'destroyedTopOuterTurret': 'mean',
    'destroyedMidOuterTurret': 'mean',
    'destroyedBotOuterTurret': 'mean',
}).round(4)

print("Mitjanes de destrucció de torres i inhibidors segons l'estat de hasWon:")

html = f"""
<div style="max-height:300px; overflow-y:auto; overflow-x:auto;
        border:1px solid #ccc; padding:6px;">
    {tower_inhibitor_stats.to_html()}
</div>
"""

display(HTML(html))

eda_df['totalTurretsDestroyed'] = (
    eda_df['destroyedTopInhibitor'] +
    eda_df['destroyedMidInhibitor'] +
    eda_df['destroyedBotInhibitor'] +
    eda_df['destroyedTopInhibitorTurret'] +
    eda_df['destroyedMidInhibitorTurret'] +
    eda_df['destroyedBotInhibitorTurret'] +
    eda_df['destroyedNexusTurrets'] +
    eda_df['destroyedTopInnerTurret'] +
    eda_df['destroyedMidInnerTurret'] +
    eda_df['destroyedBotInnerTurret'] +
    eda_df['destroyedTopOuterTurret'] +
    eda_df['destroyedMidOuterTurret'] +
    eda_df['destroyedBotOuterTurret']
)
```

```

eda_df['totalInhibitorsDestroyed'] = (
    eda_df['destroyedTopInhibitor'] +
    eda_df['destroyedMidInhibitor'] +
    eda_df['destroyedBotInhibitor']
)

# Visualització
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Mitjana d'inhibidors destruïts per resultats
ax = axes[0]
lanes = ['Top', 'Mid', 'Bot']

wins_inh = [
    eda_df[eda_df['hasWon']==1]['destroyedTopInhibitor'].mean(),
    eda_df[eda_df['hasWon']==1]['destroyedMidInhibitor'].mean(),
    eda_df[eda_df['hasWon']==1]['destroyedBotInhibitor'].mean()
]

losses_inh = [
    eda_df[eda_df['hasWon']==0]['destroyedTopInhibitor'].mean(),
    eda_df[eda_df['hasWon']==0]['destroyedMidInhibitor'].mean(),
    eda_df[eda_df['hasWon']==0]['destroyedBotInhibitor'].mean()
]

x = np.arange(len(lanes))
width = 0.35

ax.bar(x - width/2, wins_inh, width, label='Blue Team Wins', color='steelblue')
ax.bar(x + width/2, losses_inh, width, label='Blue Team Loses', color='coral')
ax.set_xlabel('Lane', fontsize=11)
ax.set_ylabel('Average Inhibitors Destroyed', fontsize=11)
ax.set_title('Inhibitors Destroyed by Lane and Outcome', fontsize=12, fontweight=
ax.set_xticks(x)
ax.set_xticklabels(lanes)
ax.legend()
ax.grid(axis='y', alpha=0.3)

# Distribució del total de torres destruïdes
ax = axes[1]

wins_data = eda_df[eda_df['hasWon']==1]['totalTurretsDestroyed']
losses_data = eda_df[eda_df['hasWon']==0]['totalTurretsDestroyed']

ax.hist(
    [losses_data, wins_data],
    bins=20,
    label=['Blue Team Loses', 'Blue Team Wins'],
    color=['coral', 'steelblue'],
    alpha=0.7
)

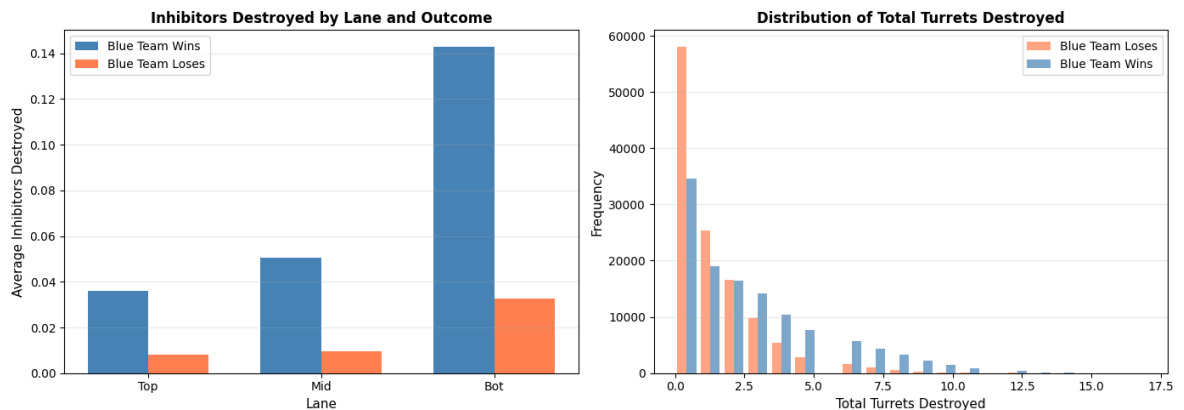
ax.set_xlabel('Total Turrets Destroyed', fontsize=11)
ax.set_ylabel('Frequency', fontsize=11)
ax.set_title('Distribution of Total Turrets Destroyed', fontsize=12, fontweight=
ax.legend()
ax.grid(axis='y', alpha=0.3)

```

```
plt.tight_layout()
plt.show()
```

Mitjanes de destrucció de torres i inhibidors segons l'estat de hasWon:

|        | destroyedTopInhibitor | destroyedMidInhibitor | destroyedBotInhibitor | destroy |
|--------|-----------------------|-----------------------|-----------------------|---------|
| hasWon |                       |                       |                       |         |
| 0      | 0.0079                | 0.0096                | 0.0328                |         |
| 1      | 0.0361                | 0.0507                | 0.1430                |         |



Quan mirem quins equips aconseguen trencar inhibidors, veiem que és un dels senyals més clars de qui acabarà guanyant la partida.

- Top: els equips guanyadors trenquen l'inhibidor unes 4 vegades més sovint que els perdedors.
- Mid: la diferència encara és més marcada, els guanyadors el destrueixen 5 cops més sovint.
- Bot: la diferència també és enorme, els guanyadors el trenquen quatre vegades més que els equips que perden.

Per tant, si un equip aconsegueix un inhibidor, és molt probable que acabi guanyant la partida. I de tots, el de la línia inferior destaca com el més decisiu: controlar aquesta línia sol marcar el resultat de la partida.

Per altra banda, els equips que guanyen destrueixen aproximadament 2,5 vegades més torres que els perdedors. Ara bé, destruir torres no fa guanyar la partida, és un resultat de tenir superioritat global, i en conseqüència aquesta superioritat es fa major.

## 4.9 Anàlisi de Visió (*Wards*)

Analitzem la importància del control de visió en la partida. Però primer de tot; abans hem vist que el màxim de *Wards* col·locats en una partida és 1014, la qual cosa és impossible. Equips professionals del joc, poden arribar a aconseguir 70 sent molt optimistes, això seria un total de 350 *Wards* per partida. Anem a veure la distribució d'aquestes dades.

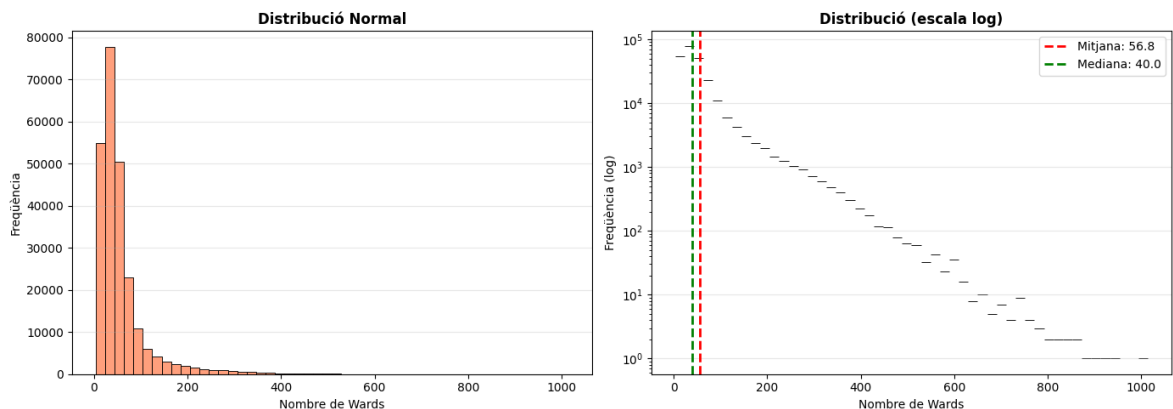
```
In [14]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```

# Sense escala Log
sns.histplot(df['wardsPlaced'], bins=50, ax=axes[0], color='coral', edgecolor='b')
axes[0].set_title('Distribució Normal', fontsize=12, fontweight='bold')
axes[0].set_xlabel('Nombre de Wards')
axes[0].set_ylabel('Freqüència')
axes[0].grid(axis='y', alpha=0.3)

# Amb escala Log
sns.histplot(df['wardsPlaced'], bins=50, log_scale=(False, True), ax=axes[1], co
axes[1].set_title('Distribució (escala log)', fontsize=12, fontweight='bold')
axes[1].set_xlabel('Nombre de Wards')
axes[1].set_ylabel('Freqüència (log)')
axes[1].grid(axis='y', alpha=0.3)
mean_val = df['wardsPlaced'].mean()
median_val = df['wardsPlaced'].median()
plt.axvline(mean_val, color='red', linestyle='--', linewidth=2, label=f'Mitjana:
plt.axvline(median_val, color='green', linestyle='--', linewidth=2, label=f'Medi
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

```



Hi ha un clúster normal entre 5–60 wards, i uns pocs valors extraordinaris (outliers) que indiquen errors. Caldrà excloure d'alguna manera aquests casos.

```

In [15]: # Calcul de mitjanes
wins_mean = [
    df[df['hasWon']==1]['wardsPlaced'].mean(),
    df[df['hasWon']==1]['wardsDestroyed'].mean(),
    df[df['hasWon']==1]['wardsLost'].mean()
]

losses_mean = [
    df[df['hasWon']==0]['wardsPlaced'].mean(),
    df[df['hasWon']==0]['wardsDestroyed'].mean(),
    df[df['hasWon']==0]['wardsLost'].mean()
]

# Visualització
plt.figure(figsize=(8, 6))

x = np.arange(3)
width = 0.35

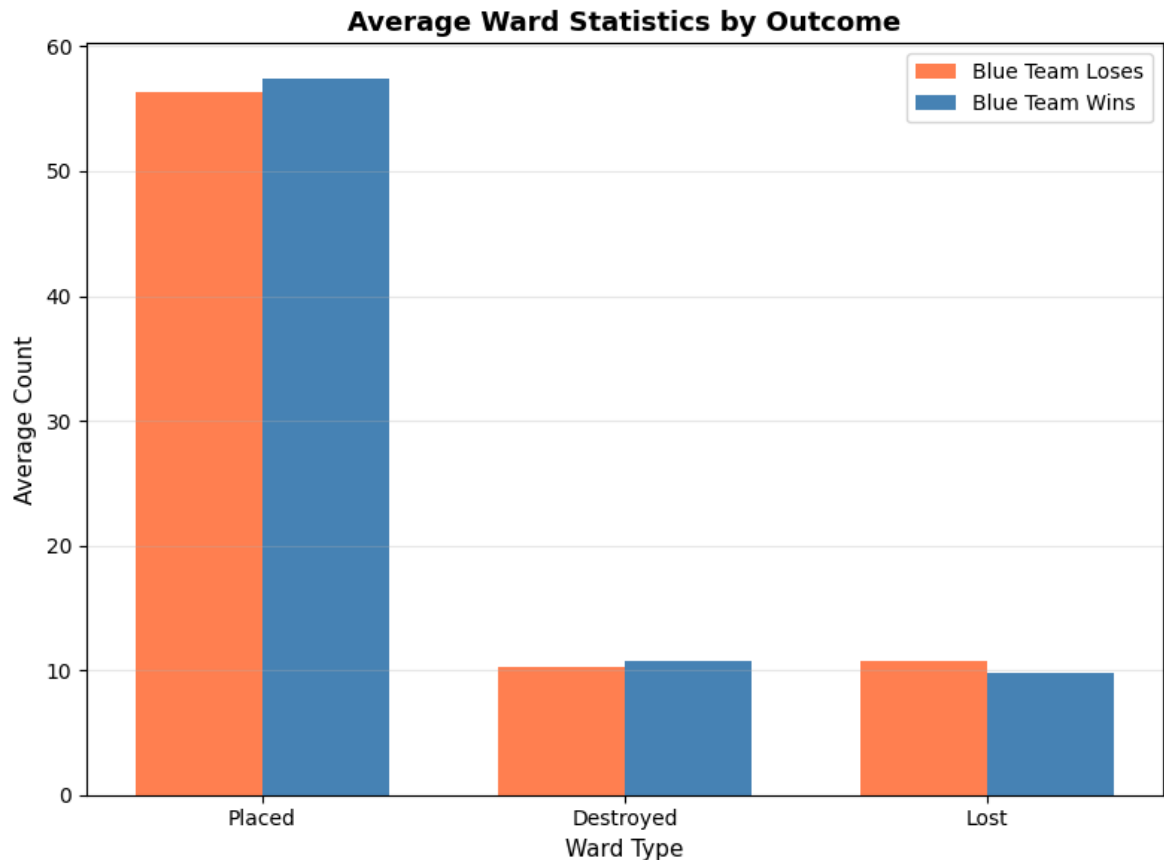
plt.bar(x - width/2, losses_mean, width, label='Blue Team Loses', color='coral')

```

```
plt.bar(x + width/2, wins_mean, width, label='Blue Team Wins', color='steelbl

plt.xlabel('Ward Type', fontsize=11)
plt.ylabel('Average Count', fontsize=11)
plt.title('Average Ward Statistics by Outcome', fontsize=13, fontweight='bold')
plt.xticks(x, ['Placed', 'Destroyed', 'Lost'])
plt.legend()
plt.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
```



Quan comparem les estadístiques de *Wards* entre equips que guanyen i equips que perden, observem que les diferències són molt petites. Tant els guanyadors com els perdedors col·loquen, destrueixen i perden pràcticament el mateix nombre de *Wards*. Els guanyadors posen només una mica més de *Wards* que els perdedors (una diferència irrellevant del 2%) i també destrueixen lleugerament més *Wards* enemics (només un 5% més) i perden una mica menys de *Wards* (un 10% menys).

Aquesta informació ens indica que la quantitat de *Wards* no és un bon indicador de victòria ja que els dos equips tenen comportaments molt similars en termes de visió.

Com a resultat, té sentit eliminar o reduir la importància d'aquestes variables per evitar soroll i millorar el rendiment del model. El motiu probablement sigui perquè l'activitat de *Wards* depèn principalment del rol *support*, que pot col·locar més quantitat alhora i com el dataset no separa rols, la informació es veu diluïda.

## 5. Preprocessament de dades i selecció de vari-

# ables

Amb tot l'anàlisi exploratori fet entenent el dataset, detectant problemes, explorant les correlacions i distribucions, podem començar amb el preprocessament de les dades.

## 5.1 Eliminació de columnes innecessàries

Abans hem arribat a la conclusió de que varies columnes no aporten informació predictiva:

- gameId: no aporta informació predictiva, només identifica la partida. Però, ens cal la variable per tal de poder agrupar les partides i que una mateixa partida no caigui en test i train.
- gameDuration: no s'hauria d'utilitzar per predir, ja que la correlació es baixa amb la variable objectiu i podria causar leakage temporal.
- frame: indica el minut, pot arribar a ser útil o no. Depenent de que volem fer amb la nostra predicció podria ser interessant mantenir-lo, per tant provarem amb i sense aquesta columna.
- expDiff: és redundant amb "champLevelDiff", eliminem aquesta ja que és menys directe que "champLevelDiff", ja que l'experiència només serveix per pujar de nivell i si no es fa servir per això, no s'hauria de tenir en compte.

```
In [16]: df_model = df.copy()

cols_to_drop = ["gameId", "gameDuration", "champLevelDiff"]

df_model = df_model.drop(columns=cols_to_drop)
```

## 5.2 Tractament dels outliers

També hem detectat el següent:

- wardsPlaced: té valors impossibles (>400)

Per tant, anem a truncar aquesta variable a un màxim de 400, perquè pot ser que el valor sigui alt, sobretot si la partida s'allarga, però no d'aquesta manera. La quantitat de partides amb una quantitat de *Wards* major a 400 és baixa, i per tant tampoc afecta al conjunt.

```
In [17]: df_model["wardsPlaced"] = df_model["wardsPlaced"].clip(upper=400)
```

## 5.3 Variables derivades

Hem arribat a la conclusió de que el nostre model és molt lineal, amb poques variables tenint una correlació alta amb la nostra variable objectiu. Per tant, la resta tenen una relació lineal molt dèbil. Fer variables derivades que ajuntin un mateix significat pot arribar a ser interessant, per tant podem provar a fer-les i veure si millora els nostres resultats.

Un d'ells seria el  $KDA = \frac{Kills + Assists}{Deaths + 1}$  per evitar la divisió per zero.

```
In [18]: df_model["kda_ratio"] = (df_model["kills"] + df_model["assists"]) / (df_model["deaths"] + 1)
df_model = df_model.drop(columns=["kills", "assists", "deaths"])
```

També, tenint en compte que a vegades no s'arriba a fer certs objectius, podria ser interessant ajuntar aquests, restant els perduts:

```
In [19]: df_model["objectiveControl"] = (
    df_model["killedFireDrake"] + df_model["killedWaterDrake"] + df_model["killedEarthDrake"] + df_model["killedElderDrake"] +
    df_model["killedBaronNashor"] + df_model["killedRiftHerald"]
    -
    (
        df_model["lostFireDrake"] + df_model["lostWaterDrake"] + df_model["lostEarthDrake"] + df_model["lostElderDrake"] +
        df_model["lostBaronNashor"] + df_model["lostRiftHerald"]
    )
)

df_model = df_model.drop(columns=["killedFireDrake", "killedWaterDrake", "killedEarthDrake", "killedElderDrake", "lostFireDrake", "lostWaterDrake", "lostEarthDrake", "lostElderDrake", "lostBaronNashor", "lostRiftHerald"])
```

Es podria fer també amb les estructures, però algunes tenen un pes més alt i per tant determinar una variable derivada d'aquesta és més complicat si no es vol perdre informació.

## 5.4 Selecció final de variables

Amb tot el nostre preprocessament de dades fet, podem definir la nostra selecció final de variables i preparar-les per els nostres models:

X = totes les variables menys hasWon (és la variable objectiu) y = hasWon

```
In [20]: X = df_model.drop(columns=["hasWon"])
y = df_model["hasWon"]
```

## 5.5 Evitar Leakage amb Group Split i GroupKFold

Com hi ha múltiples frames per partida, no podem permetre que una mateixa partida caigui en train i test. Podem utilitzar GroupShuffleSplit per crear divisions d'entrenament i prova, assegurant que totes les mostres d'un mateix grup romanguin en el mateix conjunt.

```
In [21]: gss = GroupShuffleSplit(test_size=0.2, n_splits=1, random_state=42)
groups = df["gameId"]
for train_idx, test_idx in gss.split(X, y, groups):
    X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
    y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

gkf = GroupKFold(n_splits=5)
```

# 6. Models de classificació i predicció

L'objectiu és construir models que prediguin correctament quin equip guanyarà una partida basant-se en estadístiques de joc. S'entrenen diversos models amb el les nostra selecció final de variables, es comparen i s'avaluen. Per fer-ho cal fer el següent:

1. **Línia base**
2. **Pipelines**: Cada model encapsula les transformacions necessàries (normalització, modelització...).
3. **GroupKFold**: Avaluació per evitar leakage.
4. **Mètriques d'avaluació**: Accuracy (exactitud) i ROC AUC (capacitat de discriminació).
5. **Entrenament**.
6. **Gràfics**: Matriu de confusió i curva ROC.

## 6.1 Linia base

Els models de línia base que em escollit són:

- Logistic Regression
- Ridge Classifier
- Support Vector Classifier (SVM)
- Random Forest
- HistGradientBoostingClassifier
- XGBoost Classifier
- Naive Bayes
- KNN

Aquests models cobreixen models lineals, no lineals, basats en arbres i de boosting. A part de què són útils per una classificació binària, com és el nostre cas.

## 6.2 Pipelines

Els models lineals i SVM necessiten escalat, els models d'arbres no.

```
In [22]: models = {
    "Logistic Regression": Pipeline([
        ("scaler", StandardScaler()),
        ("clf", LogisticRegression(max_iter=2000, random_state=42))
    ]),

    "Ridge Classifier": Pipeline([
        ("scaler", StandardScaler()),
        ("clf", RidgeClassifier(random_state=42))
    ]),

    "Random Forest": RandomForestClassifier(
        n_estimators=200,
        max_depth=None,
        random_state=42
    ),

    "HistGradientBoosting": HistGradientBoostingClassifier(
        learning_rate=0.1,
```

```

        max_depth=None,
        random_state=42
    ),

    "XGBoost": XGBClassifier(
        n_estimators=300,
        max_depth=6,
        learning_rate=0.1,
        subsample=0.8,
        colsample_bytree=0.8,
        objective="binary:logistic",
        eval_metric="logloss",
        random_state=42
    )
}

```

## 6.3 Avaluació amb GroupKFold

```

In [23]: scoring = {
    "accuracy": "accuracy",
    "roc_auc": "roc_auc",
    "precision": "precision",
    "recall": "recall",
    "f1": "f1"
}

cv_results = {}

for name, model in models.items():
    print(f"\nAvaluant model: {name}")

    scores = cross_validate(
        model,
        X_train,
        y_train,
        cv=gkf.split(X_train, y_train, groups=groups.iloc[train_idx]),
        scoring=scoring,
        n_jobs=-1,
        return_train_score=False
    )

    cv_results[name] = {
        "accuracy_mean": scores["test_accuracy"].mean(),
        "accuracy_std": scores["test_accuracy"].std(),
        "roc_auc_mean": scores["test_roc_auc"].mean(),
        "roc_auc_std": scores["test_roc_auc"].std(),
        "precision_mean": scores["test_precision"].mean(),
        "recall_mean": scores["test_recall"].mean(),
        "f1_mean": scores["test_f1"].mean()
    }

pd.DataFrame(cv_results).T

```

Avaluant model: Logistic Regression

Avaluant model: Ridge Classifier

Avaluant model: Random Forest

Avaluant model: HistGradientBoosting

Avaluant model: XGBoost

Out[23]:

|                             | accu-<br>racy_mean | accu-<br>racy_std | roc_auc_mean | roc_auc_std | preci-<br>sion_mean | call_ |
|-----------------------------|--------------------|-------------------|--------------|-------------|---------------------|-------|
| <b>Logistic Regression</b>  | 0.793593           | 0.003090          | 0.879483     | 0.003233    | 0.795924            | 0.7   |
| <b>Ridge Classifier</b>     | 0.790172           | 0.004219          | 0.875790     | 0.003797    | 0.791548            | 0.7   |
| <b>Random Forest</b>        | 0.788610           | 0.002980          | 0.874804     | 0.002961    | 0.784768            | 0.7   |
| <b>HistGradientBoosting</b> | 0.793603           | 0.003320          | 0.880577     | 0.003091    | 0.788034            | 0.7   |
| <b>XGBoost</b>              | 0.792130           | 0.003146          | 0.879269     | 0.003080    | 0.786497            | 0.7   |

## 6.4 Avaluació de mètriques

Amb els nostres models avaluats, ara mostrem els resultats per a poder comparar-los:

```
In [24]: cv_table = pd.DataFrame(cv_results).T  
cv_table = cv_table.sort_values(by="accuracy_mean", ascending=False)  
cv_table
```

Out[24]:

|                             | accu-<br>racy_mean | accu-<br>racy_std | roc_auc_mean | roc_auc_std | preci-<br>sion_mean | call_ |
|-----------------------------|--------------------|-------------------|--------------|-------------|---------------------|-------|
| <b>HistGradientBoosting</b> | 0.793603           | 0.003320          | 0.880577     | 0.003091    | 0.788034            | 0.7   |
| <b>Logistic Regression</b>  | 0.793593           | 0.003090          | 0.879483     | 0.003233    | 0.795924            | 0.7   |
| <b>XGBoost</b>              | 0.792130           | 0.003146          | 0.879269     | 0.003080    | 0.786497            | 0.7   |
| <b>Ridge Classifier</b>     | 0.790172           | 0.004219          | 0.875790     | 0.003797    | 0.791548            | 0.7   |
| <b>Random Forest</b>        | 0.788610           | 0.002980          | 0.874804     | 0.002961    | 0.784768            | 0.7   |

Els models obtenen resultats molt similars, ja que la predicció de victòria en Legae of Legends està altament dominada per un conjunt molt reduït de característiques, principalment la diferencia d'or i d'experiència/nivell. Aquestes variables són quasi lineals, per la qual cosa tots els models, independentment del seu tipus, convergeixen a un rendiment molt semblant (~79% accuracy i ~0.88 ROC-AUC).

Això indica que el problema és altament estructurat i quasi lineal, i que la major part de la informació predictiva és compartida i accessible per qualsevol model.

Per tant, podem actuar de forma diferent per veure si podem millorar d'alguna manera els nostres models, però primer caldrà veure si el dataset base dona millors resultats:

```

In [ ]: df2 = df.copy()
cols_to_drop = ["gameId", "gameDuration", "champLevelDiff"]
df2 = df2.drop(columns=cols_to_drop)
df2["wardsPlaced"] = df2["wardsPlaced"].clip(upper=400)

X2 = df2.drop(columns=["hasWon"])
y2 = df2["hasWon"]

gss = GroupShuffleSplit(test_size=0.2, n_splits=1, random_state=42)

for train_idx2, test_idx2 in gss.split(X2, y2, groups):
    X_train2, X_test2 = X2.iloc[train_idx2], X2.iloc[test_idx2]
    y_train2, y_test2 = y2.iloc[train_idx2], y2.iloc[test_idx2]
    groups_train2 = groups.iloc[train_idx2]

gkf = GroupKFold(n_splits=5)

cv_results2 = {}

for name, model in models.items():
    print(f"\nAvaluant model 2: {name}")

    scores = cross_validate(
        model,
        X_train2,
        y_train2,
        cv=gkf.split(X_train2, y_train2, groups=groups_train2),
        scoring=scoring,
        n_jobs=-1,
        return_train_score=False
    )

    cv_results2[name] = {
        "accuracy_mean2": scores["test_accuracy"].mean(),
        "accuracy_std2": scores["test_accuracy"].std(),
        "roc_auc_mean2": scores["test_roc_auc"].mean(),
        "roc_auc_std2": scores["test_roc_auc"].std(),
        "precision_mean2": scores["test_precision"].mean(),
        "recall_mean2": scores["test_recall"].mean(),
        "f1_mean2": scores["test_f1"].mean()
    }

pd.DataFrame(cv_results2).T

```

Avaluant model 2: Logistic Regression

Avaluant model 2: Ridge Classifier

Avaluant model 2: Random Forest

Avaluant model 2: HistGradientBoosting

Avaluant model 2: XGBoost

Out[ ]:

|                             | accu-<br>racy_mean2 | accu-<br>racy_std2 | roc_auc_mean2 | roc_auc_std2 | preci-<br>sion_mean2 |
|-----------------------------|---------------------|--------------------|---------------|--------------|----------------------|
| <b>Logistic Regression</b>  | 0.796427            | 0.004072           | 0.882959      | 0.003169     | 0.794536             |
| <b>Ridge Classifier</b>     | 0.793645            | 0.004661           | 0.879475      | 0.003712     | 0.791912             |
| <b>Random Forest</b>        | 0.791836            | 0.003339           | 0.877835      | 0.003127     | 0.789439             |
| <b>HistGradientBoosting</b> | 0.795938            | 0.003489           | 0.883513      | 0.003168     | 0.792700             |
| <b>XGBoost</b>              | 0.795211            | 0.002805           | 0.882177      | 0.003163     | 0.792113             |

```
In [ ]: results_model = pd.DataFrame(cv_results).T
results_df2 = pd.DataFrame(cv_results2).T

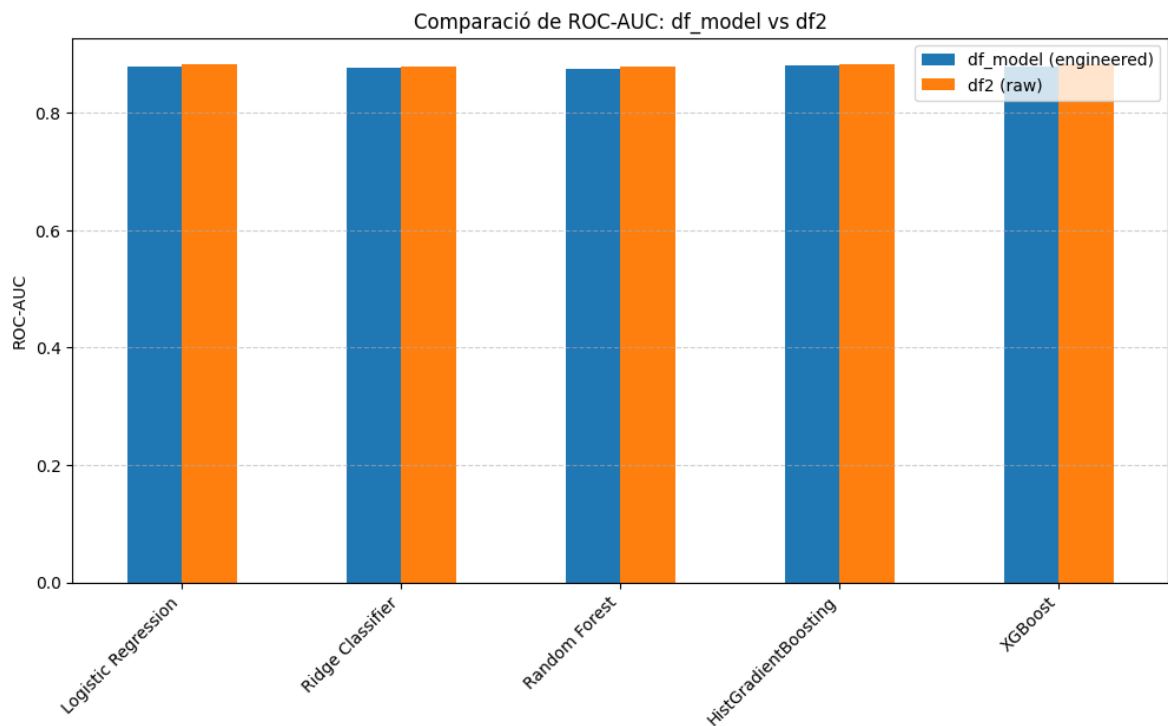
comparison = pd.concat([
    results_model[["accuracy_mean", "roc_auc_mean"]],
    results_df2[["accuracy_mean2", "roc_auc_mean2"]]
], axis=1)

plt.figure(figsize=(10,6))

comparison[["roc_auc_mean", "roc_auc_mean2"]].plot(kind="bar", figsize=(12,6))
plt.title("Comparació de ROC-AUC: df_model vs df2")
plt.ylabel("ROC-AUC")
plt.xticks(rotation=45, ha="right")
plt.legend(["df_model (engineered)", "df2 (raw)"])
plt.grid(axis="y", linestyle="--", alpha=0.6)

plt.show()
```

<Figure size 1000x600 with 0 Axes>



I veiem que dona una millora molt baixa respecte l'altre model. Probablement al fer les variables derivades hem simplificat el model, i eliminant les dades amb les que ens

basavem hem perdut detall.

```
In [ ]: df_model_results = pd.DataFrame(cv_results).T
df2_results = pd.DataFrame(cv_results2).T

worst3_df_model = df_model_results.sort_values("accuracy_mean").head(3)
worst3_df2 = df2_results.sort_values("accuracy_mean2").head(3)
print("Pitjors 3 models per df_model (engineered):")
display(worst3_df_model)

print("\nPitjors 3 models per df2 (raw):")
display(worst3_df2)
```

Pitjors 3 models per df\_model (engineered):

|                         | accu-<br>racy_mean | accu-<br>racy_std | roc_auc_mean | roc_auc_std | preci-<br>sion_mean | re-<br>call_mean | f1_me   |
|-------------------------|--------------------|-------------------|--------------|-------------|---------------------|------------------|---------|
| <b>Random Forest</b>    | 0.788610           | 0.002980          | 0.874804     | 0.002961    | 0.784768            | 0.787920         | 0.78625 |
| <b>Ridge Classifier</b> | 0.790172           | 0.004219          | 0.875790     | 0.003797    | 0.791548            | 0.780561         | 0.78597 |
| <b>XGBoost</b>          | 0.792130           | 0.003146          | 0.879269     | 0.003080    | 0.786497            | 0.794673         | 0.79052 |

Pitjors 3 models per df2 (raw):

|                         | accu-<br>racy_mean2 | accu-<br>racy_std2 | roc_auc_mean2 | roc_auc_std2 | preci-<br>sion_mean2 | re-<br>call_mean2 |
|-------------------------|---------------------|--------------------|---------------|--------------|----------------------|-------------------|
| <b>Random Forest</b>    | 0.791836            | 0.003339           | 0.877835      | 0.003127     | 0.789439             | 0.788741          |
| <b>Ridge Classifier</b> | 0.793645            | 0.004661           | 0.879475      | 0.003712     | 0.791912             | 0.789543          |
| <b>XGBoost</b>          | 0.795211            | 0.002805           | 0.882177      | 0.003163     | 0.792113             | 0.793427          |

Random Forest sempre dona els pitjors resultats, per tant podem treure'l dels models i centrar-nos en la resta.

```
In [ ]: models = {
    "Logistic Regression": Pipeline([
        ("scaler", StandardScaler()),
        ("clf", LogisticRegression(max_iter=2000, random_state=42))
    ]),

    "Ridge Classifier": Pipeline([
        ("scaler", StandardScaler()),
        ("clf", RidgeClassifier(random_state=42))
    ]),

    "HistGradientBoosting": HistGradientBoostingClassifier(
        learning_rate=0.1,
        max_depth=None,
        random_state=42
    ),

    "XGBoost": XGBClassifier(
```

```

        n_estimators=300,
        max_depth=6,
        learning_rate=0.1,
        subsample=0.8,
        colsample_bytree=0.8,
        objective="binary:logistic",
        eval_metric="logloss",
        random_state=42
    )
}

```

Intentem un altre mètode. Les partides tenen molts frames, i el model pot arribar a veure informació redundant. D'un frame al següent pot haver-hi poca diferència, per tant analitzen les mateixes dades gairebé, fent que el model massa rígida. Per tant, podem agafar el primer frame, un frame pel mig i un últim. Per tant, podem agafar el frame 10, 20 i 30. Els models haurien de variar molt més.

```

In [29]: df_10 = df[df["frame"] == 10].copy()
df_20 = df[df["frame"] == 20].copy()
df_30 = df[df["frame"] == 30].copy()

def preprocess_snapshot(df_snap):
    df_snap = df_snap.copy()

    cols_to_drop = ["gameId", "gameDuration", "champLevelDiff"]
    df_snap = df_snap.drop(columns=cols_to_drop)

    df_snap["wardsPlaced"] = df_snap["wardsPlaced"].clip(upper=400)

    X = df_snap.drop(columns=["hasWon"])
    y = df_snap["hasWon"]

    return X, y

X_10, y_10 = preprocess_snapshot(df_10)
X_20, y_20 = preprocess_snapshot(df_20)
X_30, y_30 = preprocess_snapshot(df_30)

def evaluate_snapshot(X, y, models):
    gss = GroupShuffleSplit(test_size=0.2, n_splits=1, random_state=42)
    groups = df.loc[X.index, "gameId"]

    for train_idx, test_idx in gss.split(X, y, groups):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

    results = {}
    roc_data = {}

    for name, model in models.items():

        model.fit(X_train, y_train)
        pred = model.predict(X_test)

        # Maneig
        if hasattr(model, "predict_proba"):
            prob = model.predict_proba(X_test)[: , 1]

```

```

elif hasattr(model, "decision_function"):
    decision_scores = model.decision_function(X_test)
    prob = (decision_scores - decision_scores.min()) / (decision_scores.

else:
    prob = pred

# Calcular mètriques
fpr, tpr, thresholds = roc_curve(y_test, prob)
roc_auc_val = roc_auc_score(y_test, prob)

roc_data[name] = {
    "fpr": fpr,
    "tpr": tpr,
    "roc_auc": roc_auc_val
}

results[name] = {
    "accuracy": accuracy_score(y_test, pred),
    "roc_auc": roc_auc_score(y_test, prob),
    "f1": f1_score(y_test, pred)
}

return pd.DataFrame(results).T, roc_data

results_10, roc_10 = evaluate_snapshot(X_10, y_10, models)
results_20, roc_20 = evaluate_snapshot(X_20, y_20, models)
results_30, roc_30 = evaluate_snapshot(X_30, y_30, models)

print("\nResultats Frame 10:")
display(results_10)

print("\nResultats Frame 20:")
display(results_20)

print("\nResultats Frame 30:")
display(results_30)

```

Resultats Frame 10:

|                             | accuracy | roc_auc  | f1       |
|-----------------------------|----------|----------|----------|
| <b>Logistic Regression</b>  | 0.726470 | 0.800558 | 0.735391 |
| <b>Ridge Classifier</b>     | 0.723460 | 0.799840 | 0.731279 |
| <b>HistGradientBoosting</b> | 0.725266 | 0.796207 | 0.736173 |
| <b>XGBoost</b>              | 0.704194 | 0.783299 | 0.714673 |

Resultats Frame 20:

|                             | accuracy | roc_auc  | f1       |
|-----------------------------|----------|----------|----------|
| <b>Logistic Regression</b>  | 0.805832 | 0.894068 | 0.804188 |
| <b>Ridge Classifier</b>     | 0.809366 | 0.892949 | 0.808690 |
| <b>HistGradientBoosting</b> | 0.806936 | 0.891714 | 0.805864 |
| <b>XGBoost</b>              | 0.797438 | 0.886592 | 0.796539 |

Resultats Frame 30:

|                             | accuracy | roc_auc  | f1       |
|-----------------------------|----------|----------|----------|
| <b>Logistic Regression</b>  | 0.843058 | 0.918797 | 0.836820 |
| <b>Ridge Classifier</b>     | 0.834004 | 0.916317 | 0.825212 |
| <b>HistGradientBoosting</b> | 0.830986 | 0.911689 | 0.824268 |
| <b>XGBoost</b>              | 0.826962 | 0.908596 | 0.820084 |

Deixant de banda el cross-validation, al fer una predicció per frames (temporal) es pot veure la millora dels models, i com el HistGradientBoosting va quedant-se enrere cada cop més i el XGBoost sempre queda últim.

Els resultats mostren clarament que la predictibilitat d'una partida de League of Legends augmenta dràsticament amb l'avanç de la partida.

Al minut 10, la informació encara és parcial: els models aconseguixen només un 72% d'encert.

Al minut 20, quan ja s'han disputat dracs, torres i objectius menors, la predictibilitat puja fins al 80%.

Al minut 30, amb teamfights decisives i diferències d'or consolidades, el model arriba a un 84% d'encert i una AUC superior al 0.91, indicant una separació gairebé perfecta entre equips guanyadors i perdedors.

Aquest resultat és coherent amb la dinàmica real del joc: els primers minuts són més caòtics i menys determinants, mentre que a partir del minut 20–25 el resultat sovint depèn d'un *snowball* d'objectius i avantatges acumulats.

I amb el preprocessament que havíem fet abans, milloren els resultats?

```
In [30]: df_10_2 = df[df["frame"] == 10].copy()
df_20_2 = df[df["frame"] == 20].copy()
df_30_2 = df[df["frame"] == 30].copy()

def preprocess_snapshot(df_snap):
    df_snap = df_snap.copy()

    cols_to_drop = ["gameId", "gameDuration", "champLevelDiff"]
    df_snap = df_snap.drop(columns=cols_to_drop)

    df_snap["wardsPlaced"] = df_snap["wardsPlaced"].clip(upper=400)

    # KDA
    df_snap["kda_ratio"] = (df_snap["kills"] + df_snap["assists"]) / (df_snap["deaths"] + 1)
    df_snap = df_snap.drop(columns=["kills", "assists", "deaths"])

    # Objectius
    df_snap["objectiveControl"] = (
        df_snap["killedFireDrake"] + df_snap["killedWaterDrake"] + df_snap["killedMountainDrake"] +
        df_snap["killedEarthDrake"] + df_snap["killedElderDrake"] +
        df_snap["killedBaronNashor"] + df_snap["killedRiftHerald"]
    )
```

```

        (
            df_snap["lostFireDrake"] + df_snap["lostWaterDrake"] + df_snap["lost
            df_snap["lostEarthDrake"] + df_snap["lostElderDrake"] +
            df_snap["lostBaronNashor"] + df_snap["lostRiftHerald"]
        )
    )

df_snap = df_snap.drop(columns=[
    "killedFireDrake", "killedWaterDrake", "killedAirDrake",
    "killedEarthDrake", "killedElderDrake", "killedBaronNashor", "killedRift
    "lostFireDrake", "lostWaterDrake", "lostAirDrake",
    "lostEarthDrake", "lostElderDrake", "lostBaronNashor", "lostRiftHerald"
])

X = df_snap.drop(columns=["hasWon"])
y = df_snap["hasWon"]

return X, y

X_10_2, y_10_2 = preprocess_snapshot(df_10)
X_20_2, y_20_2 = preprocess_snapshot(df_20)
X_30_2, y_30_2 = preprocess_snapshot(df_30)

def evaluate_snapshot(X, y, models):
    gss = GroupShuffleSplit(test_size=0.2, n_splits=1, random_state=42)
    groups = df.loc[X.index, "gameId"]

    for train_idx, test_idx in gss.split(X, y, groups):
        X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
        y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

    results = {}

    for name, model in models.items():

        model.fit(X_train, y_train)
        pred = model.predict(X_test)

        # Maneig
        if hasattr(model, "predict_proba"):
            prob = model.predict_proba(X_test)[:, 1]

        elif hasattr(model, "decision_function"):
            decision_scores = model.decision_function(X_test)
            prob = (decision_scores - decision_scores.min()) / (decision_scores.

        else:
            prob = pred

        # Calcular mètriques
        results[name] = {
            "accuracy_2": accuracy_score(y_test, pred),
            "roc_auc_2": roc_auc_score(y_test, prob),
            "f1_2": f1_score(y_test, pred)
        }

    return pd.DataFrame(results).T

results_10_2 = evaluate_snapshot(X_10_2, y_10_2, models)
results_20_2 = evaluate_snapshot(X_20_2, y_20_2, models)

```

```

results_30_2 = evaluate_snapshot(X_30_2, y_30_2, models)

print("\nResultats Frame 10 (2):")
display(results_10_2)

print("\nResultats Frame 20 (2):")
display(results_20_2)

print("\nResultats Frame 30 (2):")
display(results_30_2)

```

Resultats Frame 10 (2):

|                             | accuracy_2 | roc_auc_2 | f1_2     |
|-----------------------------|------------|-----------|----------|
| <b>Logistic Regression</b>  | 0.726470   | 0.800423  | 0.735699 |
| <b>Ridge Classifier</b>     | 0.723058   | 0.799588  | 0.732351 |
| <b>HistGradientBoosting</b> | 0.728075   | 0.798992  | 0.739573 |
| <b>XGBoost</b>              | 0.712222   | 0.786036  | 0.724549 |

Resultats Frame 20 (2):

|                             | accuracy_2 | roc_auc_2 | f1_2     |
|-----------------------------|------------|-----------|----------|
| <b>Logistic Regression</b>  | 0.806715   | 0.892509  | 0.805253 |
| <b>Ridge Classifier</b>     | 0.808262   | 0.891211  | 0.807111 |
| <b>HistGradientBoosting</b> | 0.803844   | 0.889759  | 0.802842 |
| <b>XGBoost</b>              | 0.795670   | 0.883740  | 0.794764 |

Resultats Frame 30 (2):

|                             | accuracy_2 | roc_auc_2 | f1_2     |
|-----------------------------|------------|-----------|----------|
| <b>Logistic Regression</b>  | 0.837022   | 0.913504  | 0.829114 |
| <b>Ridge Classifier</b>     | 0.836016   | 0.911291  | 0.827878 |
| <b>HistGradientBoosting</b> | 0.825453   | 0.907037  | 0.816693 |
| <b>XGBoost</b>              | 0.821429   | 0.903093  | 0.812665 |

El nostre preprocessament empitjora els resultats. Ens quedem amb el model per frames sense preprocessament.

Anem a veure les seves curves ROC per ajudar a prendre una decisió:

```

In [31]: def plot_roc_from_data(roc_data, frame_label):
            plt.figure(figsize=(8,6))

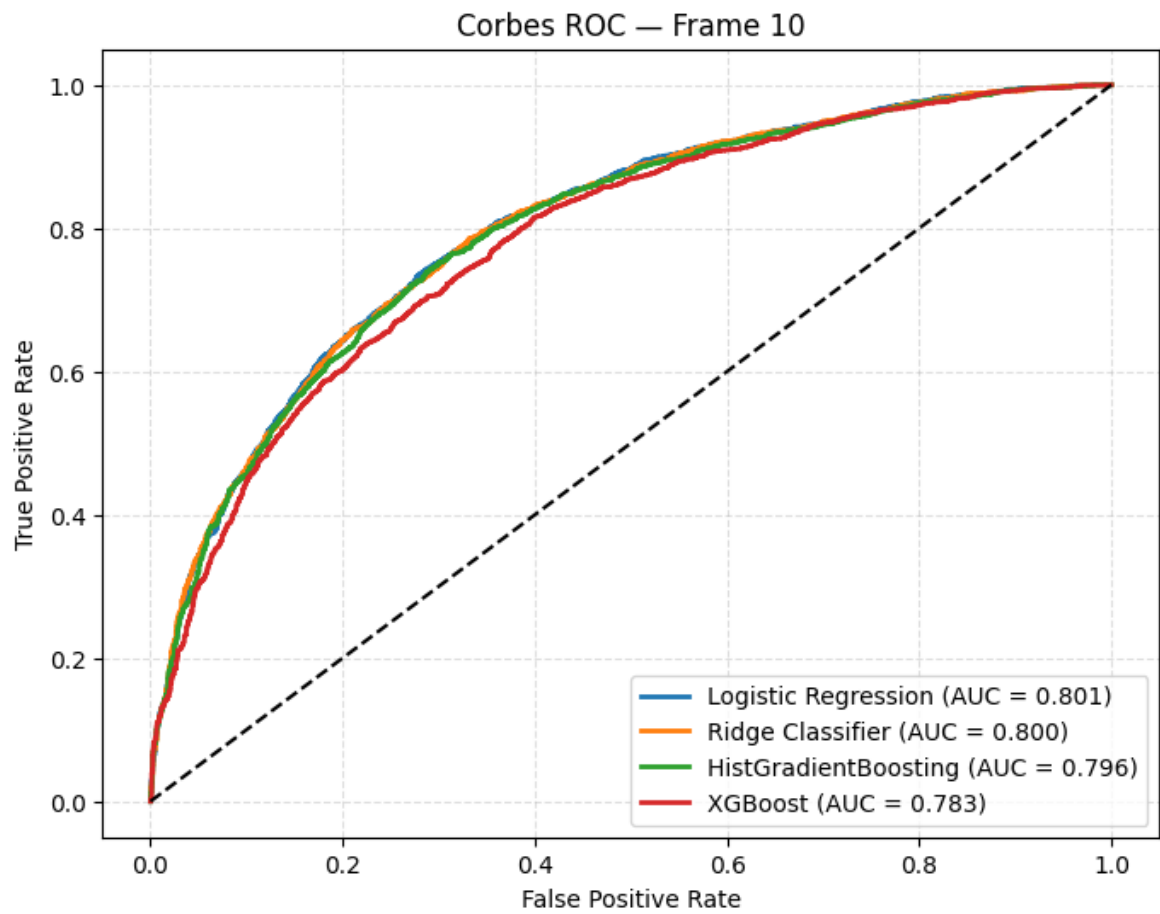
            for name, values in roc_data.items():
                plt.plot(values["fpr"], values["tpr"], lw=2, label=f"{name} (AUC = {valu

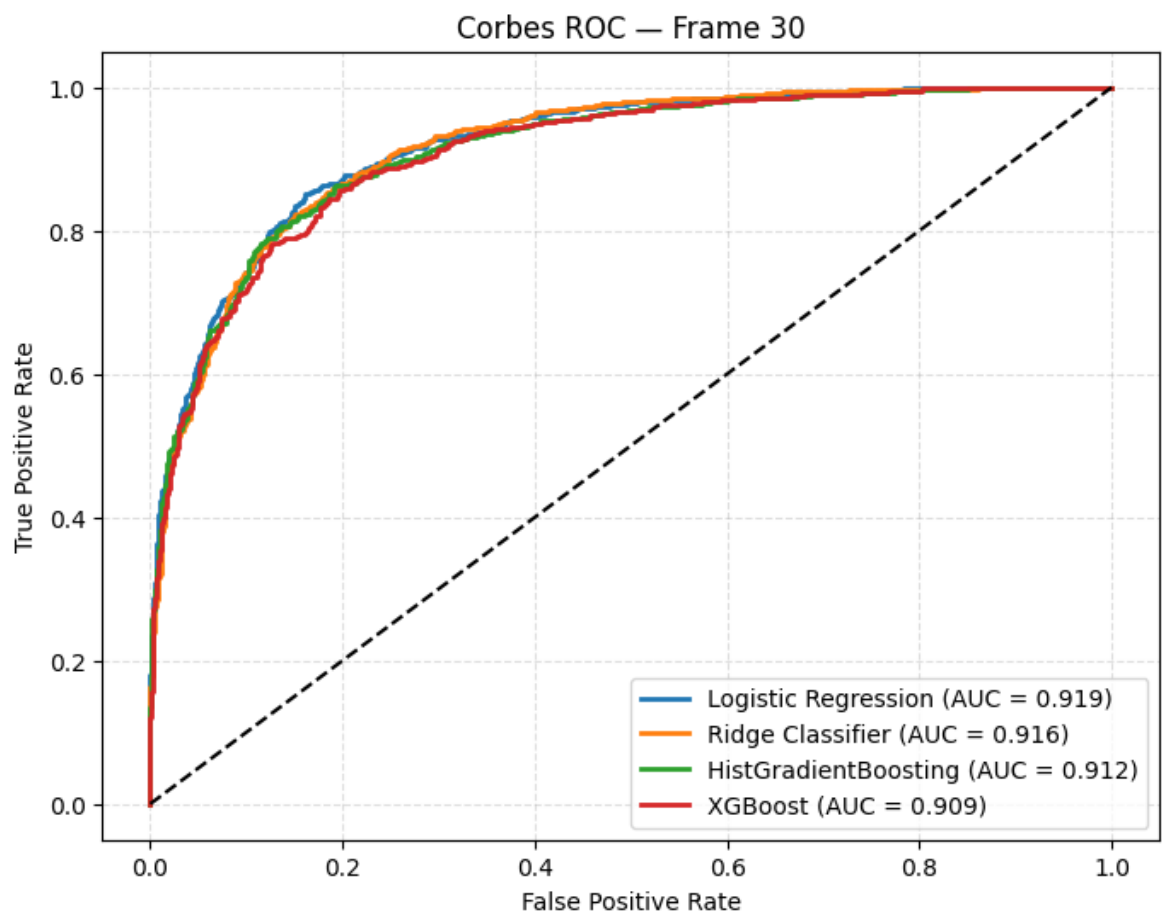
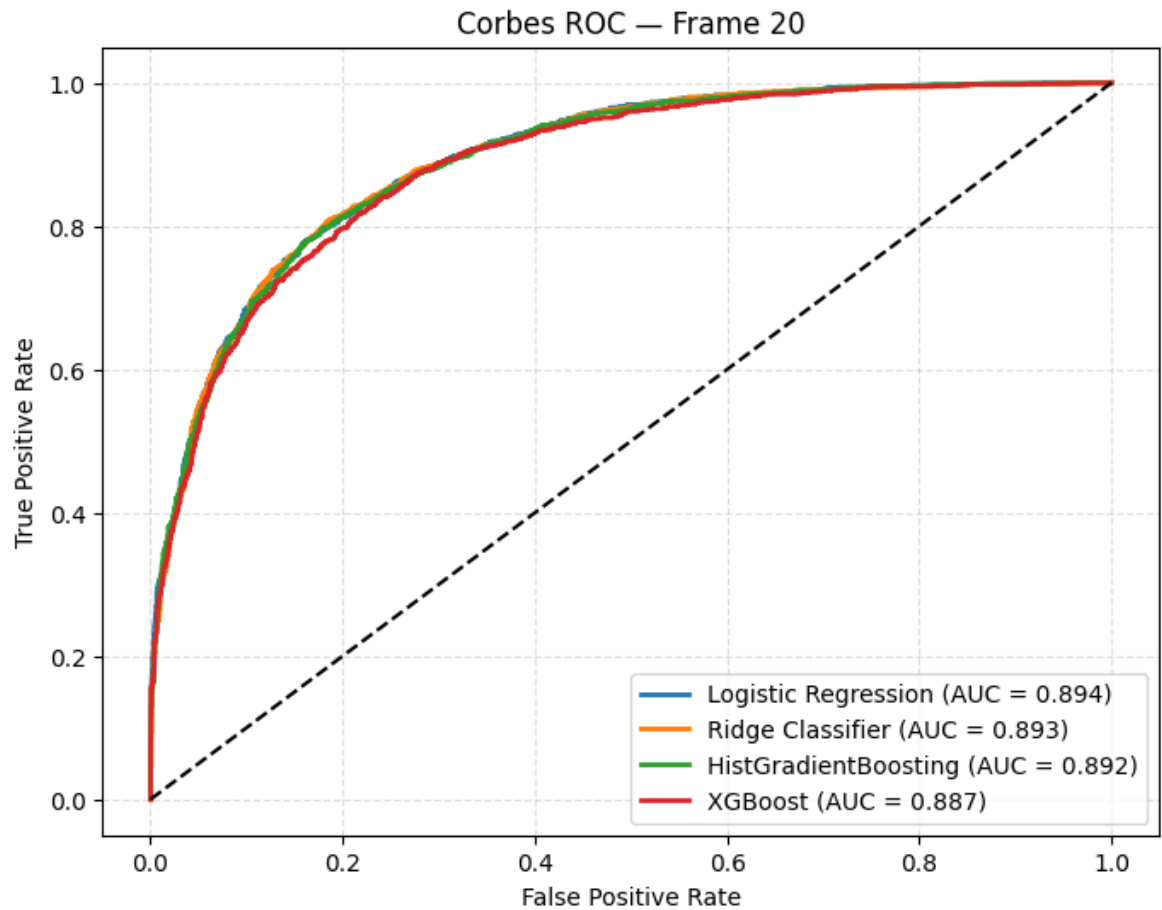
            plt.plot([0,1],[0,1], 'k--')
            plt.title(f"Corbes ROC - Frame {frame_label}")
            plt.xlabel("False Positive Rate")
            plt.ylabel("True Positive Rate")

```

```
plt.legend()  
plt.grid(True, linestyle="--", alpha=0.4)  
plt.show()
```

```
plot_roc_from_data(roc_10, 10)  
plot_roc_from_data(roc_20, 20)  
plot_roc_from_data(roc_30, 30)
```





Tot i que els models es comporten de manera semblant, Logistic Regression sempre esta una mica per sobre. Amb aquesta informació, escollim aquest model.

## 6.5 Entrenament del millor model

Per a trobar els millors hiperparàmetres per a la Regressió Logística, fem servir GridSearchCV. Teòricament, pel nostre tipus de problema, fer servir RandomizedSearchCV seria més eficient, però com el millor model ha sortit la Regressió Logística, GridSearch és més adequat i eficient perquè l'espai d'hiperparàmetres és petit.

```
In [ ]: # Cerca d'hiperparàmetres per a La Regresió Logística
param_grid = {
    "clf__C": [0.001, 0.01, 0.1, 1, 10, 100],
    "clf__penalty": ["l2"],
    "clf__solver": ["liblinear", "lbfgs"],
}

lr_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=5000, random_state=42))
])

gkf = GroupKFold(n_splits=5)

def tune_lr(X, y):
    lr_pipeline = Pipeline([
        ("scaler", StandardScaler()),
        ("clf", LogisticRegression(max_iter=5000, random_state=42))
    ])

    param_grid = {
        "clf__C": [0.001, 0.01, 0.1, 1, 10, 100],
        "clf__solver": ["liblinear", "lbfgs"],
        "clf__penalty": ["l2"]
    }

    groups = df.loc[X.index, "gameId"]
    gkf = GroupKFold(n_splits=5)

    grid = GridSearchCV(
        lr_pipeline,
        param_grid,
        cv=gkf.split(X, y, groups),
        scoring="roc_auc",
        n_jobs=-1
    )

    grid.fit(X, y)
    return grid

best_lr_10 = tune_lr(X_10, y_10)
best_lr_20 = tune_lr(X_20, y_20)
best_lr_30 = tune_lr(X_30, y_30)

print("Best params Frame 10:", best_lr_10.best_params_)
print("Best ROC-AUC 10:", best_lr_10.best_score_)

print("Best params Frame 20:", best_lr_20.best_params_)
print("Best ROC-AUC 20:", best_lr_20.best_score_)
```

```
print("Best params Frame 30:", best_lr_30.best_params_)
print("Best ROC-AUC 30:", best_lr_30.best_score_)
```

```
Best params Frame 10: {'clf__C': 0.1, 'clf__penalty': 'l2', 'clf__solver': 'lbfgs'}
Best ROC-AUC 10: 0.8033504426745397
Best params Frame 20: {'clf__C': 0.01, 'clf__penalty': 'l2', 'clf__solver': 'liblinear'}
Best ROC-AUC 20: 0.8985718010675441
Best params Frame 30: {'clf__C': 0.1, 'clf__penalty': 'l2', 'clf__solver': 'liblinear'}
Best ROC-AUC 30: 0.9205494340499232
```

Fent servir la cerca d'hiperparàmetres, hem trobar que:

Frame 10: C=0.1, solver=lbfgs.

Frame 20: C=0.01, solver=liblinear.

Frame 30: C=0.1, solver=liblinear.

## 6.6 Gràfics (visualització dels resultats)

```
In [42]: # Crear DataFrame amb els resultats
best_params_df = pd.DataFrame({
    "Frame": [10, 20, 30],
    "Best C": [
        best_lr_10.best_params_["clf__C"],
        best_lr_20.best_params_["clf__C"],
        best_lr_30.best_params_["clf__C"]
    ],
    "Best Solver": [
        best_lr_10.best_params_["clf__solver"],
        best_lr_20.best_params_["clf__solver"],
        best_lr_30.best_params_["clf__solver"]
    ],
    "Best ROC-AUC": [
        best_lr_10.best_score_,
        best_lr_20.best_score_,
        best_lr_30.best_score_
    ]
})

# Mapping dels solvers a números per fer-los gràfics
solver_map = {"lbfgs": 0, "liblinear": 1}
solver_numeric = [solver_map[s] for s in best_params_df["Best Solver"]]

# Crear figura amb 3 subplots
plt.figure(figsize=(15, 5))

# ROC-AUC per Frame
plt.subplot(1, 3, 1)
plt.bar(best_params_df["Frame"], best_params_df["Best ROC-AUC"], color="skyblue")
plt.ylim(0.75, 1.0)
plt.title("Millor ROC-AUC per Frame")
plt.xlabel("Frame")
plt.ylabel("ROC-AUC")
plt.grid(axis="y", linestyle="--", alpha=0.4)
```

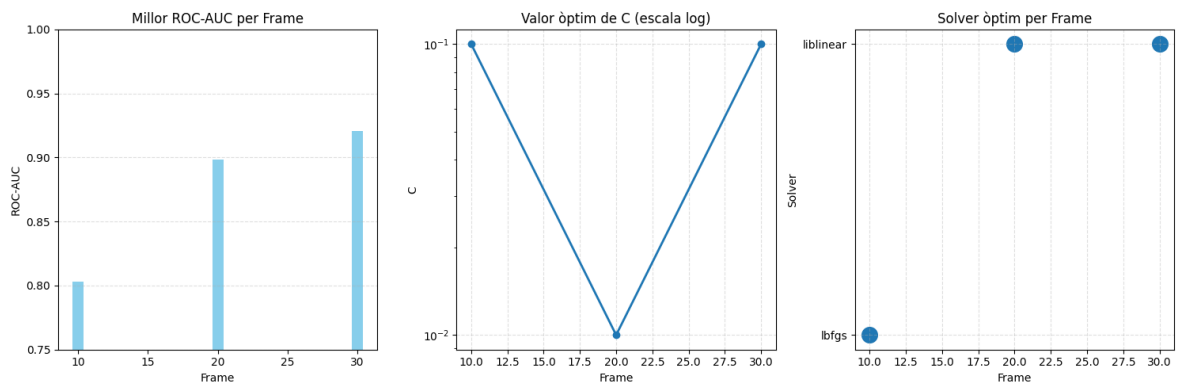
```

# C per Frame (escala log)
plt.subplot(1, 3, 2)
plt.plot(best_params_df["Frame"], best_params_df["Best C"], marker="o", linewidth=2)
plt.yscale("log")
plt.title("Valor òptim de C (escala log)")
plt.xlabel("Frame")
plt.ylabel("C")
plt.grid(True, linestyle="--", alpha=0.4)

# Solver òptim per Frame
plt.subplot(1, 3, 3)
plt.scatter(best_params_df["Frame"], solver_numeric, s=200)
plt.yticks([0,1], ["lbfgs", "liblinear"])
plt.title("Solver òptim per Frame")
plt.xlabel("Frame")
plt.ylabel("Solver")
plt.grid(True, linestyle="--", alpha=0.4)

plt.tight_layout()
plt.show()

```



Els hiperparàmetres òptims de la Regressió Logística revelen una tendència coherent amb la dinàmica del joc.

- Frame 10 —  $C = 0.1$ , solver = lbfgs A aquesta fase inicial, el joc encara és molt variable i poc determinista. Un valor C petit (més regularització) ajuda a evitar sobreajustament davant fluctuacions aleatòries.
- Frame 20 —  $C = 0.01$ , solver = liblinear Les estadístiques comencen a clarificar-se, però encara existeix una gran variabilitat en els combats i objectius. La millor configuració és una regularització forta ( $C=0.01$ ), que evita que el model sobreinterpreti diferències que encara poden ser reversibles.
- Frame 30 —  $C = 0.1$ , solver = liblinear En aquesta fase, la partida ja mostra diferències clares entre equips. La necessitat de regularització disminueix, i un C moderadament baix (0.1) permet capturar patrons més marcats sense sobreajustar.

En conjunt, aquest comportament mostra com la predictibilitat del resultat augmenta a mesura que avança la partida, reduint la necessitat de regularitzar agressivament els models.

## 7. Conclusions

L'objectiu d'aquest estudi ha estat analitzar fins a quin punt és possible predir el resultat d'una partida de League of Legends utilitzant estadístiques objectives obtingudes en diferents instants temporals del joc. El dataset analitzat inclou informació segmentada per frames, fet que requeria una estratègia de validació creuada per evitar *data leakage* i assegurar la robustesa dels models. Per aquest motiu, totes les particions entre entrenament i validació s'han realitzat mitjançant esquemes basats en grups (GroupShuffleSplit i GroupKFold), garantint que tots els frames d'una mateixa partida quedessin sempre en el mateix conjunt.

Pel que fa al preprocessament, es van examinar diverses estratègies de selecció i transformació de variables. Si bé inicialment es van generar variables derivades (com la relació KDA o un índex agregat de control d'objectius), els resultats empírics van mostrar que aquestes transformacions no aportaven una millora consistent en el rendiment predictiu. En alguns casos, fins i tot, degradaven lleugerament la capacitat dels models.

Els resultats obtinguts mostren una coherència significativa entre els diferents algoritmes analitzats. Tant els models lineals (Logistic Regression i Ridge), com els models d'arbres (RandomForest) i els basats en boosting (HistGradientBoosting i XGBoost), presenten un rendiment similar quan s'entrenen amb els frames entre 10 i 30 minuts. Les mètriques de ROC-AUC s'incrementen progressivament des d'aproximadament 0.80 al minut 10 fins a valors propers a 0.92 al minut 30, confirmant que la predictibilitat del resultat augmenta a mesura que el joc evoluciona i les diferències d'estat entre equips es fan més pronunciades. De totes maneres, la Regressió Logística és el model més robust, i per tant ha sigut escollit com el millor classificador pel nostre problema.

Justament l'anàlisi d'aquest model ha revelat també una evolució coherent entre els diferents instants de temps, suggerint que el model necessita una regularització més intensa per tractar la variabilitat i incertesa al principi del joc, mentres que a mesura que avança la partida, la necessitat de regularització disminueix i el model pot capturar relacions més marcades entre els atributs i el resultat final.

Tot i els bons resultats, cal reconèixer que els models conserven un marge d'error no negligible. Aquest fet és perfectament esperable, ja que el desenvolupament d'una partida de League of Legends depèn de múltiples factors no recollits en el dataset: composició d'equips, estil de joc, habilitat individual, execució en situacions d'alta tensió, etc. Aquests elements, sovint qualitatius o contextuais, no són capturables mitjançant estadístiques numèriques estàtiques i, per tant, imposen un límit natural a la capacitat predictiva del model.

En conclusió, aquest estudi confirma que és viable predir el resultat d'una partida utilitzant dades parcials i sense conèixer l'estat final del joc. Els models desenvolupats mostren un comportament estable i consistent, i ofereixen una base sòlida per a futurs treballs.